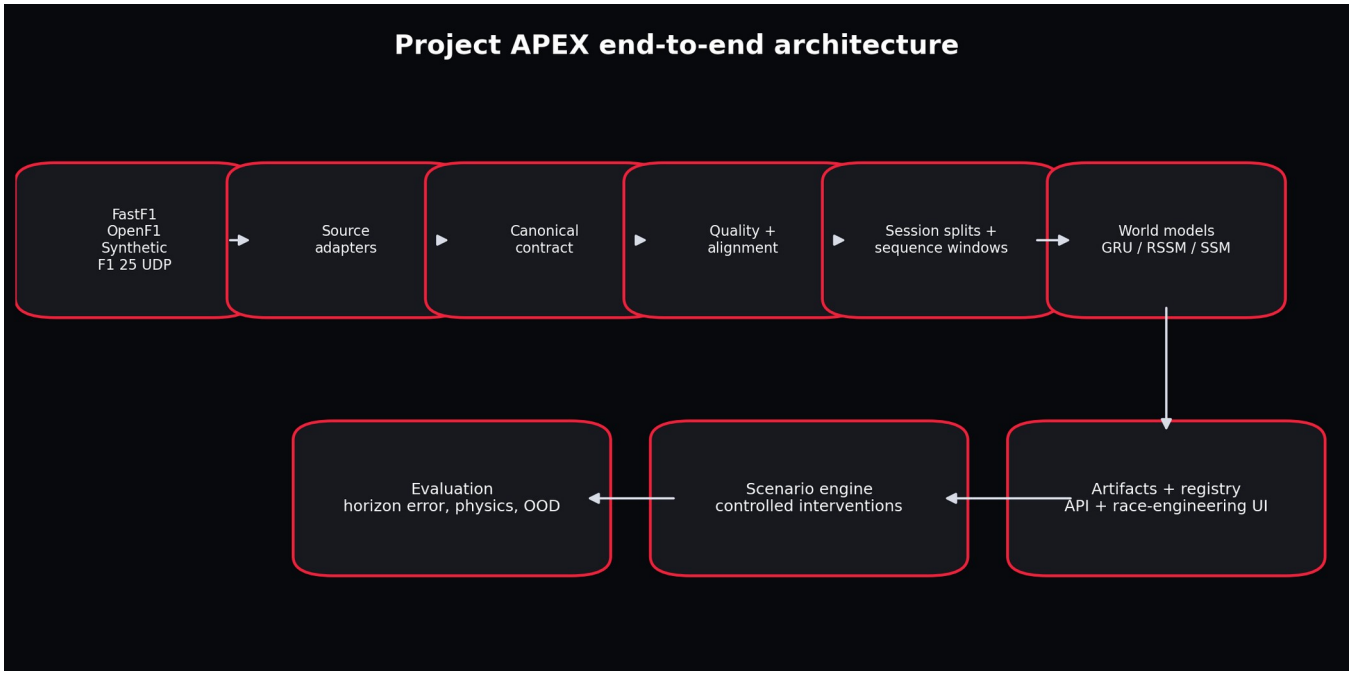
# PROJECT APEX

## THE ULTIMATE F1 WORLD-SIMULATION ENGINE TEXTBOOK

From first principles to FastF1/OpenF1 ingestion, GRU/RSSM/SSM world models, production pipelines, evaluation, fine-tuning, and a working interactive V1

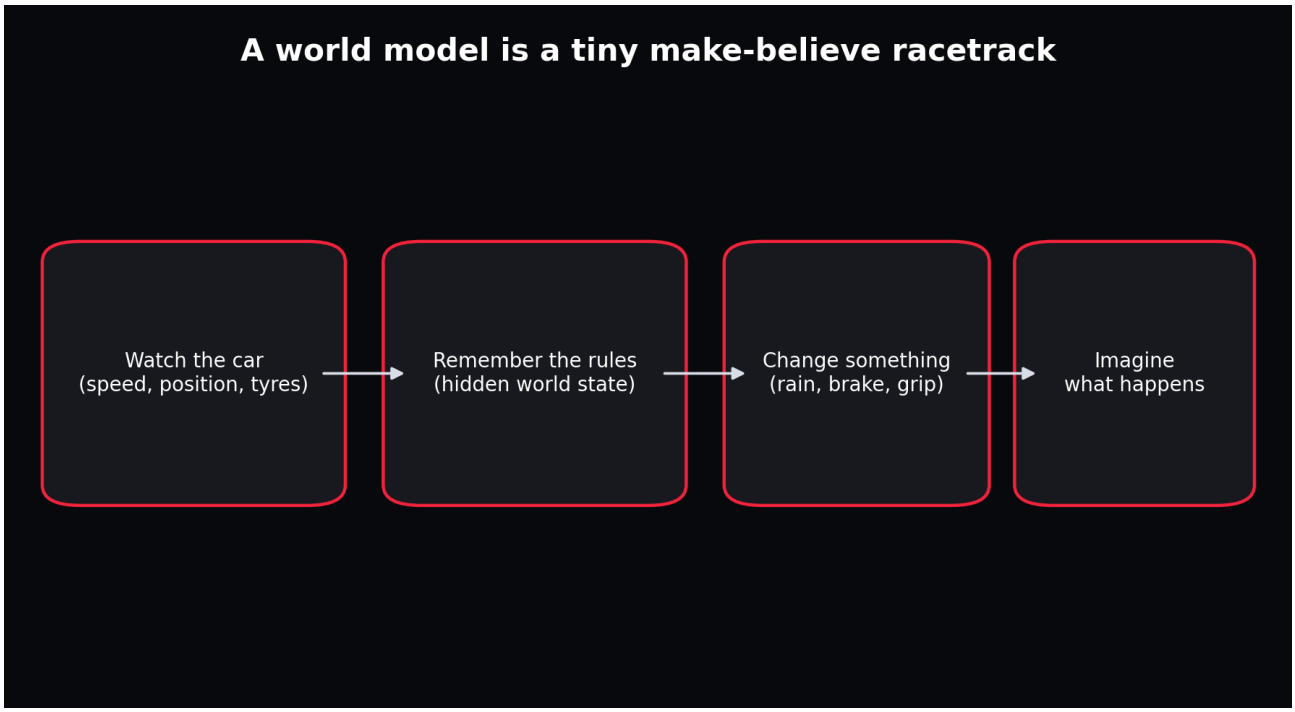



*The complete learning path and production architecture.*

**Instructor Edition | Beginner to Production**

# How to Use This Book

This is not a book to skim and then declare understood. It is a build-along course, engineering reference, and debugging manual. The chapters first build the mental machinery required for the project. The later parts ask you to predict outputs, run code, inspect artifacts, break assumptions, compare implementations, and defend decisions.




*The child-level explanation and the production system describe the same idea at different resolutions.*

| Mode | What you do | Evidence of learning |
|---|---|---|
| Read | Study the concept and redraw the visual from memory. | You can explain it without copying the wording. |
| Run | Execute the associated notebook or command. | You can predict shapes, files and approximate behaviour before execution. |
| Break | Introduce the listed defect. | You can identify which invariant was violated and where detection belongs. |
| Choose | Compare alternatives using the decision table. | You can state why one design is appropriate now and what future evidence would justify another. |
| Rebuild | Implement the component without viewing the solution. | Tests pass and your implementation preserves the same contract. |

# Table of Contents

# Part I - First-Principles Foundations

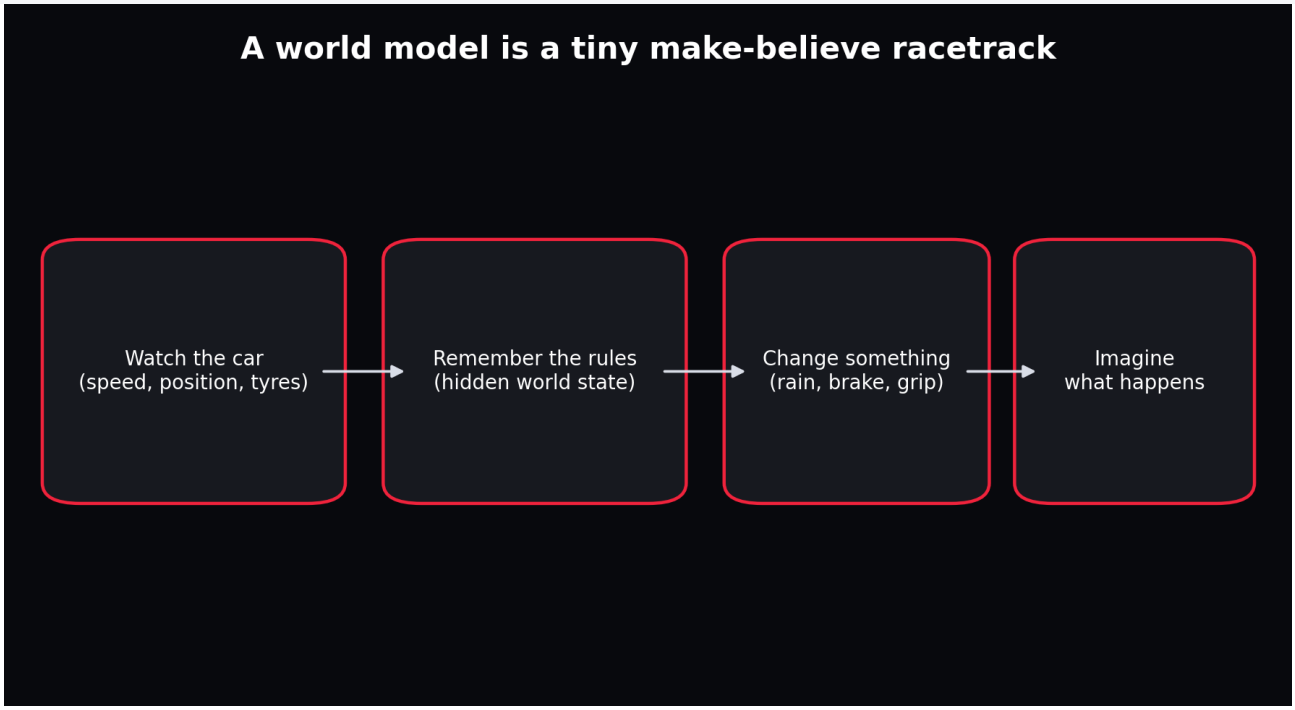
The purpose of this part is not to turn you into a mechanical engineer before you write Python. It is to give every variable, loss and model output a physical or statistical meaning. Code becomes easy to reason about when you can tell a coherent story about the quantities moving through it.

# Chapter 1 - Explain the Entire Project to a Five-Year-Old

**WHAT YOU ARE LEARNING:** The simplest truthful explanation of a world model and the exact boundary of Project APEX.

**WHY IT MATTERS:** If the simple explanation and the technical architecture disagree, the technical system is probably confused.

**YOU ARE FINISHED WHEN:** You can explain the project using toys and then translate every toy into an engineering term.




*A world model observes, remembers, accepts a change, and imagines a future.*

Imagine a toy car on a track. You watch it for a while. You notice that pressing the accelerator usually makes it faster, pressing the brake makes it slower, tight corners require lower speed, rain

makes the track slippery, and old tyres gradually lose grip. Now imagine building a little brain that remembers those patterns. When you ask, "What happens if it rains more?", the brain draws a possible next few seconds.

The toy car is the environment. What you can measure is the observation. The compact description of what matters now is the state. Driver controls are actions. Track and weather variables are context. The learned rule that moves the current state into the next state is the transition model. Repeatedly applying the transition is a rollout or imagination.

**SCOPE BOUNDARY:** V1 is a telemetry dynamics simulator. It is not a complete rigid-body physics engine, a licensed F1 game, or an autonomous racing agent. It learns short future telemetry trajectories from recorded-style data.

**INSTRUCTOR CHECKPOINT:** Close the book and explain the system to a child in four sentences. Then explain it to an ML engineer using state, action, context, transition and rollout.

# Chapter 2 - What Is a Simulation Engine?

**WHAT YOU ARE LEARNING:** The difference among replay, forecasting, simulation, physics modelling and control.

**WHY IT MATTERS:** Many projects call any chart with a prediction a simulator. That produces exaggerated claims and poor evaluation.

**YOU ARE FINISHED WHEN:** You can classify a component correctly and state what evidence would upgrade it to the next class.

| System | Input | Output | Can answer interventions? |
|---|---|---|---|
| Replay | Recorded data | The same recorded trajectory | No |
| Forecast | Past observations | Likely future observations | Only weakly |
| Data-driven simulator | Past state plus explicit future actions/context | A conditional future trajectory | Yes, within support |
| Physics simulator | Physical parameters and controls | Numerically integrated physical state | Yes, under model assumptions |
| Controller/planner | State, objective and simulator | Selected actions | Yes, by searching or learning |

Project APEX begins as a data-driven simulator. It is conditioned on planned control/context trajectories. Its counterfactual claims are credible only near combinations represented in training.

A throttle multiplier of 1.05 is a modest intervention; inventing zero grip at 350 km/h is far outside the learned world.

### Worked example

> **QUESTION:** A dashboard predicts the next lap time from previous lap times but has no input for tyre choice or weather changes. Is it a simulator?

Step 1. Identify whether the model receives causes that a user can deliberately change.

Step 2. Notice that it predicts an outcome from history but cannot condition on a proposed intervention.

Step 3. Classify it as a forecast and describe how to make it simulation-like.

> **ANSWER:** It is a forecasting model. Add explicit state, action and context variables, then evaluate controlled interventions against held-out examples or a trusted physical simulator.

# Chapter 3 - Units, Sampling and Dimensional Thinking

> **WHAT YOU ARE LEARNING:** SI units, conversions, rates, discrete sampling and dimensional checks.
>
> **WHY IT MATTERS:** Most telemetry bugs are not neural-network bugs. They are km/h versus m/s, milliseconds versus seconds, percentages versus fractions, or mismatched sample rates.
>
> **YOU ARE FINISHED WHEN:** You can detect a unit error from magnitude and verify an equation dimensionally.

The canonical contract uses metres, seconds, metres per second, metres per second squared, degrees Celsius, and unit fractions for controls. Source adapters perform conversion once. Model code should never ask whether Speed means km/h or m/s.

| Quantity | Canonical unit | Common source representation | Conversion |
|---|---|---|---|
| Speed | m/s | km/h | divide by 3.6 |
| Throttle | 0 to 1 | 0 to 100 percent | divide by 100 |
| Time | seconds | timedelta or ISO timestamp | convert to elapsed seconds |
| Gear | integer plus normalized fraction | integer 0-8 | gear_norm = gear / 8 |
| Rainfall | 0 to 1 severity proxy | boolean or category | define and document adapter mapping |

```
speed_kmh = 288.0
speed_mps = speed_kmh / 3.6
sample_hz = 5
dt = 1.0 / sample_hz
distance_one_frame = speed_mps * dt
print(speed_mps, distance_one_frame)
```

*At 288 km/h, the car travels 16 metres during one 5 Hz frame.*

### Worked example

> **QUESTION:** A car moves at 270 km/h. How far does it travel between samples at 4 Hz?

Step 1. Convert 270 km/h to m/s: 270 / 3.6 = 75 m/s.

Step 2. At 4 Hz, one frame lasts 1/4 = 0.25 s.

Step 3. Distance = speed x time = 75 x 0.25.

> **ANSWER:** 18.75 metres. This large spacing explains why independently sampled location and control streams require careful alignment.

# Chapter 4 - Kinematics Without Fear

> **WHAT YOU ARE LEARNING:** Position, velocity, acceleration, discrete differences and integration.
>
> **WHY IT MATTERS:** The predicted channels must obey a coherent motion story.
>
> **YOU ARE FINISHED WHEN:** You can compute acceleration from frames and explain why differencing amplifies noise.

Velocity describes how position changes. Acceleration describes how velocity changes. In sampled telemetry, acceleration is commonly estimated as (v_t - v_(t-1)) / dt. This estimate magnifies measurement noise because a small speed error is divided by a small time interval. Smooth only with a documented causal filter when the model will run online.

```
dt = timestamp_s.diff()
acceleration = speed_mps.diff() / dt
# Never allow zero or negative dt to pass silently.
```

> **COMMON FAILURE:** Interpolating across a long data gap creates a smooth but imaginary acceleration profile. Mark large gaps and split the episode instead of fabricating continuity.

# Chapter 5 - Forces and a Useful Toy Vehicle Model

**WHAT YOU ARE LEARNING:** Engine force, braking, aerodynamic drag, corner demand, wet grip and tyre degradation.

**WHY IT MATTERS:** The offline generator needs causal structure strong enough to test the learning system.

**YOU ARE FINISHED WHEN:** You can explain every term in the synthetic acceleration equation and identify what it omits.

The generator uses a deliberately simple longitudinal equation: acceleration equals engine contribution minus braking, aerodynamic drag, corner demand and wet penalty, plus small noise. It is not a CFD or tyre model. It is a controlled world where changing throttle, brake, curvature, rain and grip produces the correct directional effects.

```
acceleration = (
   engine_accel
   - brake_accel
   - aero_drag
   - corner_drag
   - wet_drag
   + stochastic_force
)
speed_next = clip(speed_now + acceleration * dt, 8.0, 101.0)
```

| Term | Teaching role | Missing real-world detail |
|---|---|---|
| engine_accel | Throttle causes positive acceleration | Power curve, ERS, gear ratios, traction limits |
| brake_accel | Brake causes deceleration | Brake temperature, balance, lock-up |
| aero_drag | Penalty grows with speed squared | Ride height, wings, DRS maps, yaw |
| corner_drag | Curvature limits desired speed | Lateral tyre force and load transfer |
| wet_drag/grip | Weather changes dynamics | Water depth, tyre compound, aquaplaning |

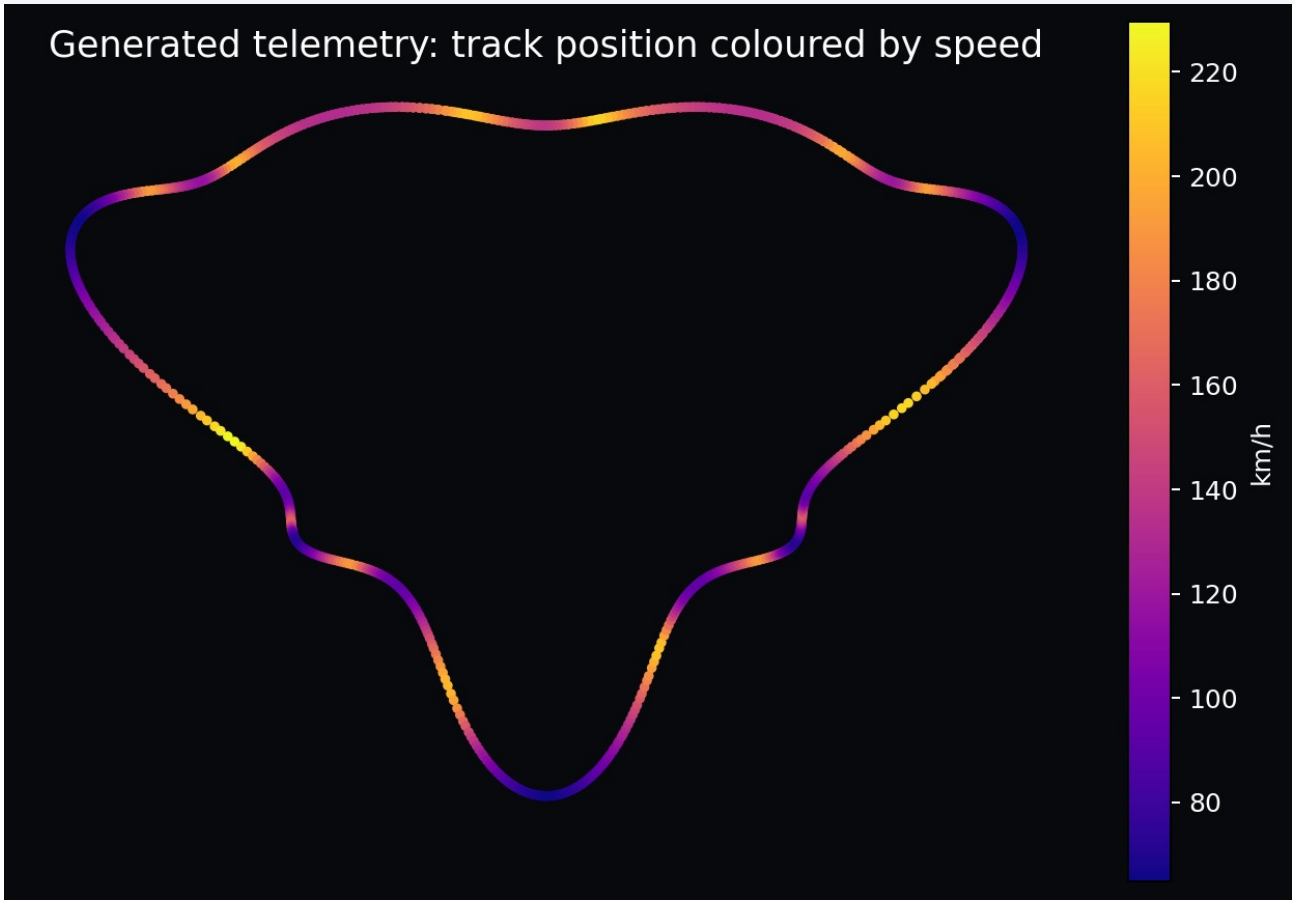
**INSTRUCTOR CHECKPOINT:** State two reasons a learned model can outperform this toy equation on its own generated data, and two reasons that would not prove better real-world physics.

# Chapter 6 - Track Geometry, Progress and Curvature

**WHAT YOU ARE LEARNING:** Cartesian position, heading, curvature, cyclic progress and track maps.

**WHY IT MATTERS:** The same speed means different things on a straight and at a hairpin.

**YOU ARE FINISHED WHEN:** You can encode cyclic progress without a discontinuity at the finish line.




*A generated closed track; colour reveals the causal relationship between geometry and speed.*

Raw progress jumps from almost 1 back to 0 at the finish line. A neural model may interpret that as a violent discontinuity. Encode progress as sin(2 pi p) and cos(2 pi p). Points just before and after the line are then close on the unit circle.

```
progress_sin = np.sin(2 * np.pi * progress)
progress_cos = np.cos(2 * np.pi * progress)
# Recover angle with atan2(progress_sin, progress_cos).
```

### Worked example

> **QUESTION:** Progress changes from 0.99 to 0.01. Why is raw mean squared error misleading?

Step 1. Raw difference is 0.98 even though the car moved a tiny distance across the start line.

Step 2. The representation ignores circular topology.

Step 3. Map both values to nearby points on a unit circle.

> **ANSWER:** Use sine/cosine encoding or a circular loss. Also monitor whether predicted sine/cosine remain near unit norm.

# Chapter 7 - Probability as Multiple Possible Futures

> **WHAT YOU ARE LEARNING:** Random variables, conditional distributions, expectation, variance and uncertainty.
>
> **WHY IT MATTERS:** Traffic, driver decisions, grip and measurement noise create futures that are not perfectly deterministic.
>
> **YOU ARE FINISHED WHEN:** You can distinguish noise, model uncertainty and scenario uncertainty.

A deterministic model returns one trajectory. A probabilistic world model represents a distribution p(s_future | history, actions, context). The mean is not automatically the most useful output. At a braking point with two plausible driver behaviours, the mean trajectory may resemble neither behaviour.

| Uncertainty | Example | Treatment |
|---|---|---|
| Aleatoric | Small grip fluctuations or measurement noise | Predict a distribution or multiple samples |
| Epistemic | Unseen wet track or new driver | Ensembles, OOD detection, more data |
| Scenario | Different planned actions | Condition explicitly and compare rollouts |
| Data quality | Missing or misaligned packets | Quality flags; do not disguise as model uncertainty |

# Chapter 8 - Statistics for Telemetry

**WHAT YOU ARE LEARNING:** Mean, variance, covariance, correlation, conditional analysis and train-only normalization.

**WHY IT MATTERS:** Telemetry features have different scales and strong dependence.

**YOU ARE FINISHED WHEN:** You can calculate a standardized value and explain why global normalization leaks information.

Standardization transforms x into (x - mean_train) / std_train. Fit statistics on training sessions only. A mean computed using test weather or test-driver speed distribution transfers information from evaluation into training.

### Worked example

**QUESTION:** Training speed has mean 62 m/s and standard deviation 12 m/s. What is the standardized value of 86 m/s?

Step 1. Subtract the training mean: 86 - 62 = 24.

Step 2. Divide by training standard deviation: 24 / 12.

**ANSWER:** The standardized speed is 2.0. It is two training standard deviations above the training mean.

**CORRELATION WARNING:** Throttle and speed may be positively correlated on straights and negatively related immediately before corners because drivers lift before speed falls. Time direction and context matter.

# Chapter 9 - Vectors, Matrices and Tensors as Organized Measurements

**WHAT YOU ARE LEARNING:** Feature vectors, batches, sequence axes, matrix multiplication and tensor semantics.

**WHY IT MATTERS:** Deep-learning code becomes English when every axis has a sentence.

**YOU ARE FINISHED WHEN:** You can annotate [B,T,F] and catch an accidental axis swap.

For Project APEX, a history tensor has shape [batch, time, features]. Batch groups independent examples. Time preserves causal order. Features describe one instant. A model that receives [time, batch, features] when it expects batch first may still run and silently learn nonsense.

```
history.shape == [B, T_history, F_input]
future_inputs.shape == [B, T_future, F_input]
future_targets.shape == [B, T_future, F_state]
```

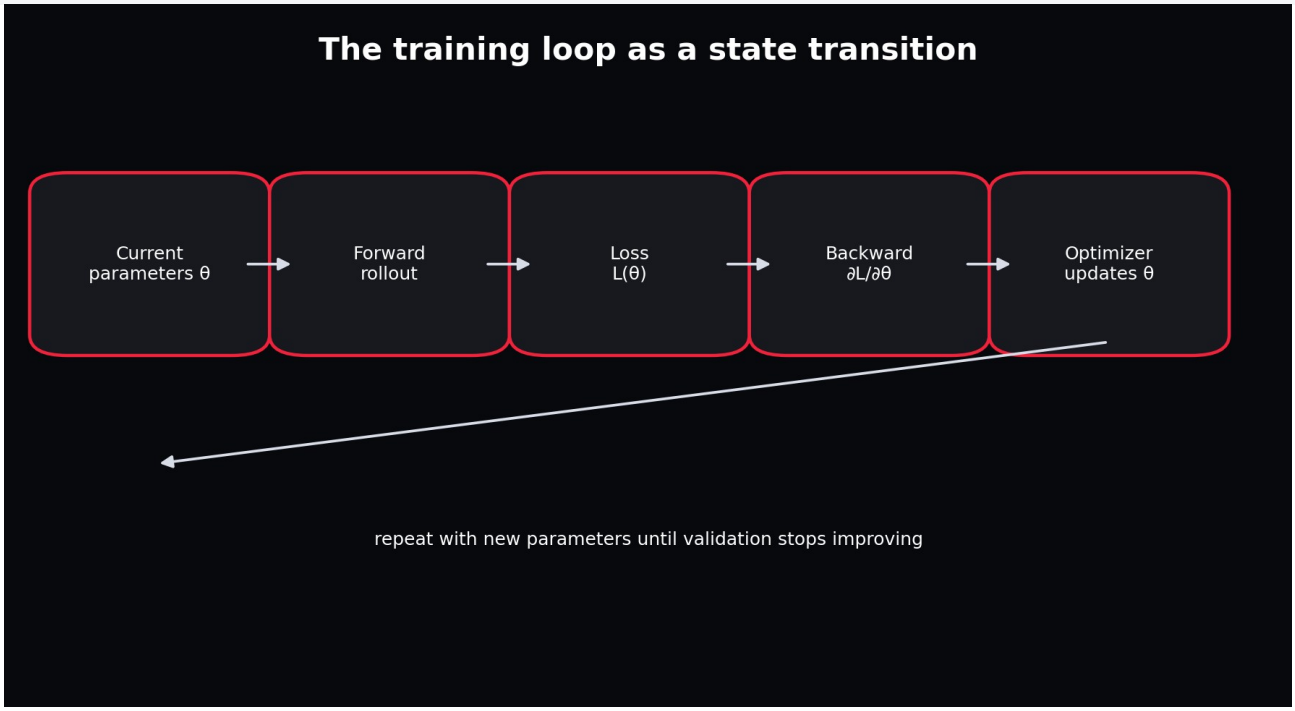
**INSTRUCTOR CHECKPOINT:** For a batch of 64 windows, 32 history frames, 16 input features and an 8-step future with 5 targets, write all three tensor shapes without looking.

# Chapter 10 - Gradients and Optimization

**WHAT YOU ARE LEARNING:** Loss functions, derivatives, backpropagation, learning rate, clipping and validation.

**WHY IT MATTERS:** Training is a controlled numerical procedure, not a magical call to fit.

**YOU ARE FINISHED WHEN:** You can narrate each line of the training loop and diagnose a loss that does not decrease.




*Training repeatedly changes parameters using gradients computed from prediction error.*

The gradient is a local sensitivity: if a parameter changes slightly, how does the loss change? Backpropagation applies the chain rule through every operation. The optimizer uses those

sensitivities to update parameters. A learning rate that is too large can overshoot; one that is too small can look frozen. Gradient clipping limits an unstable update but should not hide a broken loss or scale.

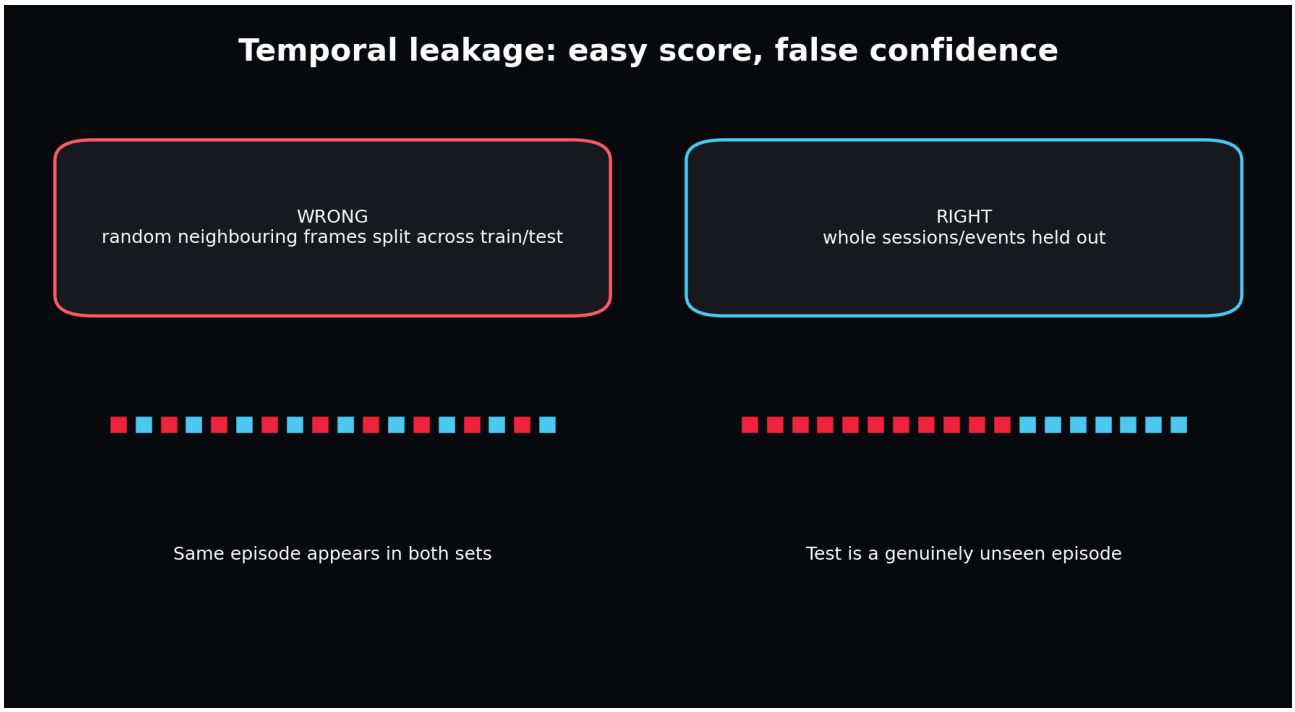
```
optimizer.zero_grad(set_to_none=True)
prediction = model(history, future_inputs)
loss = smooth_l1_loss(prediction, future_targets)
loss.backward()
clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

# Part II - Sequence Models and State-Space Models

# Chapter 11 - Time Series Are Not Shuffled Tables

> **WHAT YOU ARE LEARNING:** Autocorrelation, non-stationarity, episodes, gaps and causal order.
>
> **WHY IT MATTERS:** Random tabular assumptions produce leakage and unrealistic validation.
>
> **YOU ARE FINISHED WHEN:** You can design an episode-aware split and explain what future information is allowed.

Adjacent telemetry frames are highly similar. Weather and tyre state evolve slowly. A random row split creates test examples almost duplicated in training. Project APEX splits whole sessions, then creates windows that never cross session boundaries.




*Neighbour leakage creates flattering metrics that collapse on genuinely new sessions.*

# Chapter 12 - Markov State and Partial Observability

> **WHAT YOU ARE LEARNING:** The Markov property, hidden variables and belief state.
>
> **WHY IT MATTERS:** A single telemetry row rarely contains everything required to predict the future.
>
> **YOU ARE FINISHED WHEN:** You can explain why history is needed even when current speed and controls are known.

A state is Markov if the next state depends on the present state and action, not the entire past. Real telemetry is partially observed: tyre temperature, driver intent, exact grip and energy state may be hidden. A sequence model uses history to build a belief state that summarizes those hidden causes.

## Worked example

> **QUESTION:** Two cars have identical speed, throttle and track position. One has overheated tyres and the other does not. Are the visible rows Markov?

Step 1. The future differs because an omitted variable changes available grip.

Step 2. Current visible features are therefore insufficient.

Step 3. History may reveal the hidden condition through recent slip, pace or temperature proxies.

**ANSWER:** No. The observation is partially observable. Add the missing sensor if possible or use recurrent history to estimate a belief state.

# Chapter 13 - RNNs: A Memory That Updates One Step at a Time

**WHAT YOU ARE LEARNING:** Recurrent hidden states, unrolling and vanishing/exploding gradients.

**WHY IT MATTERS:** RNNs establish the basic pattern used by GRUs, RSSMs and recurrent SSM inference.

**YOU ARE FINISHED WHEN:** You can write the recurrence and draw an unrolled sequence.

A simple RNN updates $h_t = \tanh(W_x x_t + W_h h_{(t-1)} + b)$. The same parameters are reused at every time step. This weight sharing lets it process variable-length streams. Long chains of derivatives can vanish or explode, motivating gates and structured dynamics.

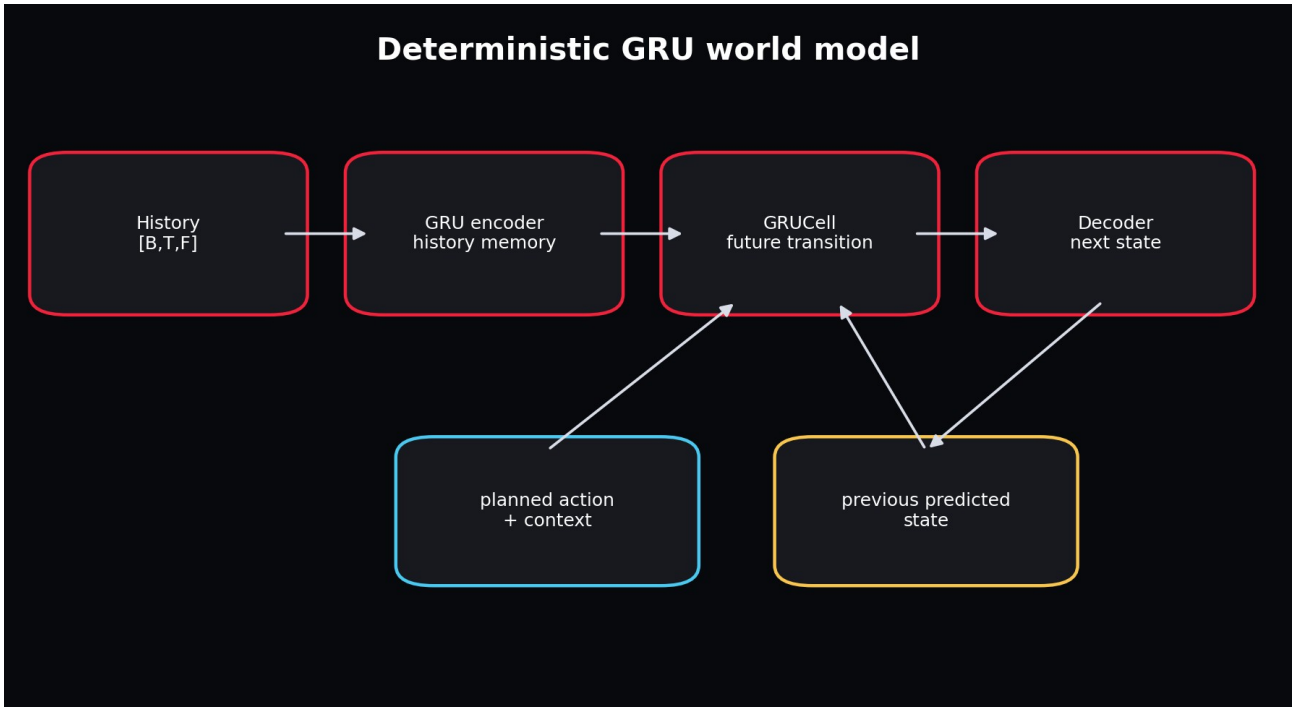
# Chapter 14 - GRUs: Gated, Practical Memory

**WHAT YOU ARE LEARNING:** Update gates, reset gates and a deterministic sequence baseline.

**WHY IT MATTERS:** The GRU is the safest first neural world model for this project.

**YOU ARE FINISHED WHEN:** You can explain what the gate controls without memorizing every symbol.

The update gate chooses how much previous memory to keep. The reset gate controls how much old memory participates in a new candidate. In practice, PyTorch implements these equations efficiently. Your job is to understand the input/output contract and rollout semantics.



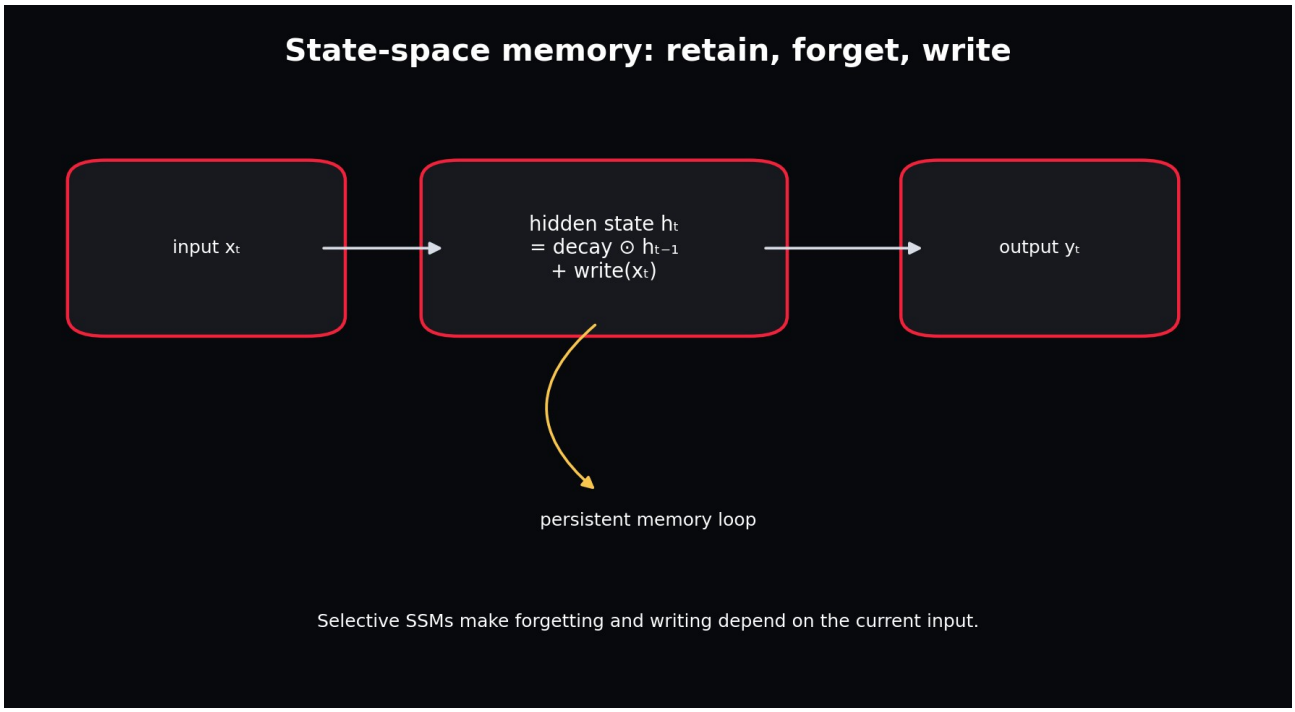

*The history encoder creates memory; the transition cell rolls forward using planned controls and previous predictions.*

> **WHY GRU FIRST:** It is causal, compact, available in core PyTorch, easy to profile, and strong enough to reveal data-contract problems.

# Chapter 15 - Classical State-Space Models

> **WHAT YOU ARE LEARNING:** Hidden state, transition matrix, input matrix and observation matrix.
>
> **WHY IT MATTERS:** Modern SSMs inherit a powerful continuous-time modelling perspective.
>
> **YOU ARE FINISHED WHEN:** You can map A, B and C to memory, input writing and output reading.

A continuous linear state-space model can be written $h'(t) = A\,h(t) + B\,x(t)$, $y(t) = C\,h(t) + D\,x(t)$. Discretization turns it into a recurrence. A controls memory dynamics, B writes inputs, C reads the hidden state, and D provides a direct path. The hidden state can be much richer than the observed output.




*A state-space model keeps a persistent state rather than recomputing all pairwise interactions.*

# Chapter 16 - S4 and Structured Long-Range Memory

**WHAT YOU ARE LEARNING:** Why naive SSMs are hard to train and how structure enables efficient sequence modelling.

**WHY IT MATTERS:** S4 is the bridge from classical control models to modern deep sequence architectures.

**YOU ARE FINISHED WHEN:** You can state the engineering advantage without deriving the Cauchy kernel.

S4 parameterizes the state matrix so long-range dynamics remain stable and can be computed efficiently. During training, an SSM can often be represented as a convolution across a known sequence; during streaming inference, it can run as a recurrence with fixed-size memory. This duality is attractive for long telemetry histories.

**DO NOT OVERCLAIM:** The included APEX selective SSM cell teaches the recurrence and gating idea. It does not reproduce the full S4 mathematical parameterization or optimized kernels.

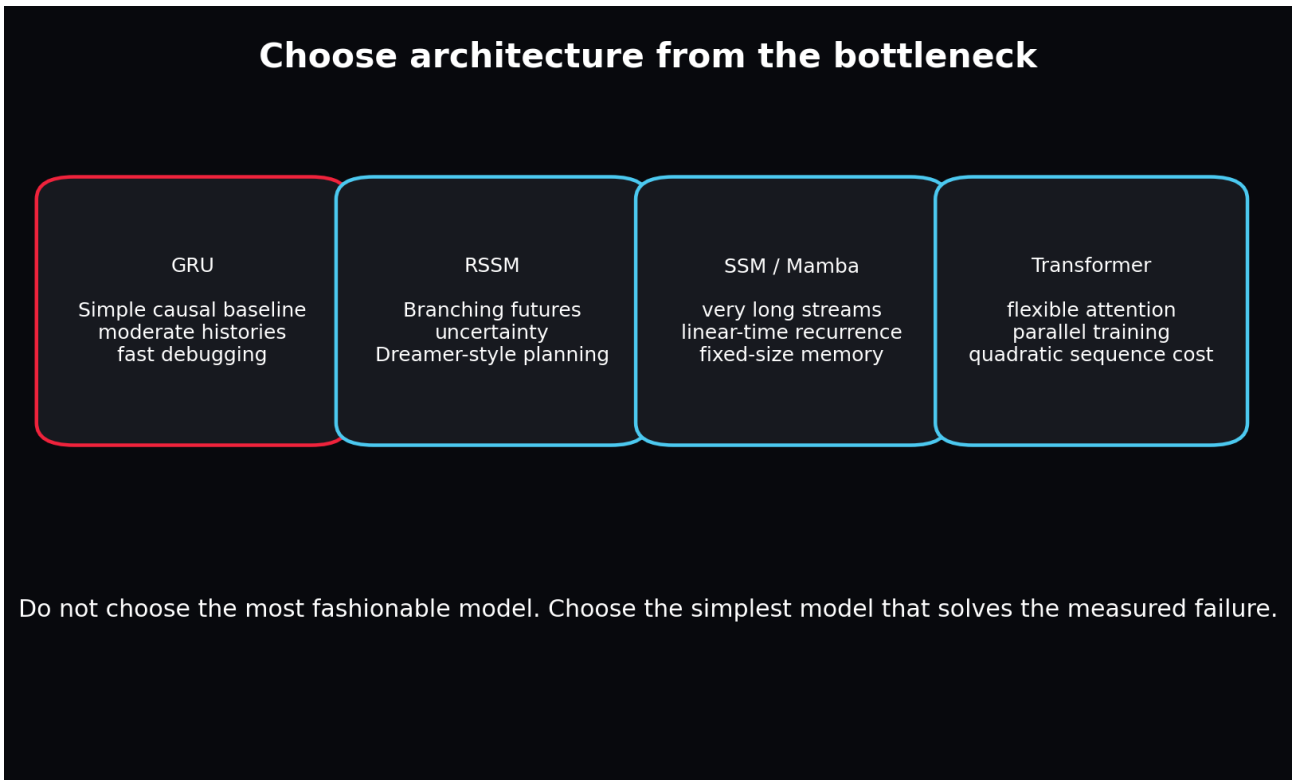
# Chapter 17 - Mamba and Selective State Spaces

**WHAT YOU ARE LEARNING:** Input-dependent retention, linear sequence scaling and recurrent inference.

**WHY IT MATTERS:** Telemetry contains occasional events that deserve selective memory: braking zones, pit entry, rain onset, damage and safety cars.

**YOU ARE FINISHED WHEN:** You can explain why input-dependent parameters are more expressive than a time-invariant recurrence.

Traditional linear time-invariant SSM dynamics treat every token with the same transition rule. Mamba makes selected state-space parameters functions of the input, allowing the model to retain or forget information based on current content. The published architecture also uses a hardware-aware scan for efficient training and inference.




*Architecture should be selected from the measured bottleneck, not prestige.*

| Use case | Prefer | Reason |
|---|---|---|
| Short, medium telemetry windows and first implementation | GRU | Lowest debugging and dependency cost |
| Long streaming histories with bounded recurrent state | SSM/Mamba | Linear sequence scaling and efficient recurrent inference |
| Multiple plausible futures and | RSSM | Stochastic latent imagination |

| | | |
|---|---|---|
| uncertainty | | |
| Flexible pairwise interactions in moderate sequences | Transformer | Direct attention across positions |

# Chapter 18 - Transformers in Context

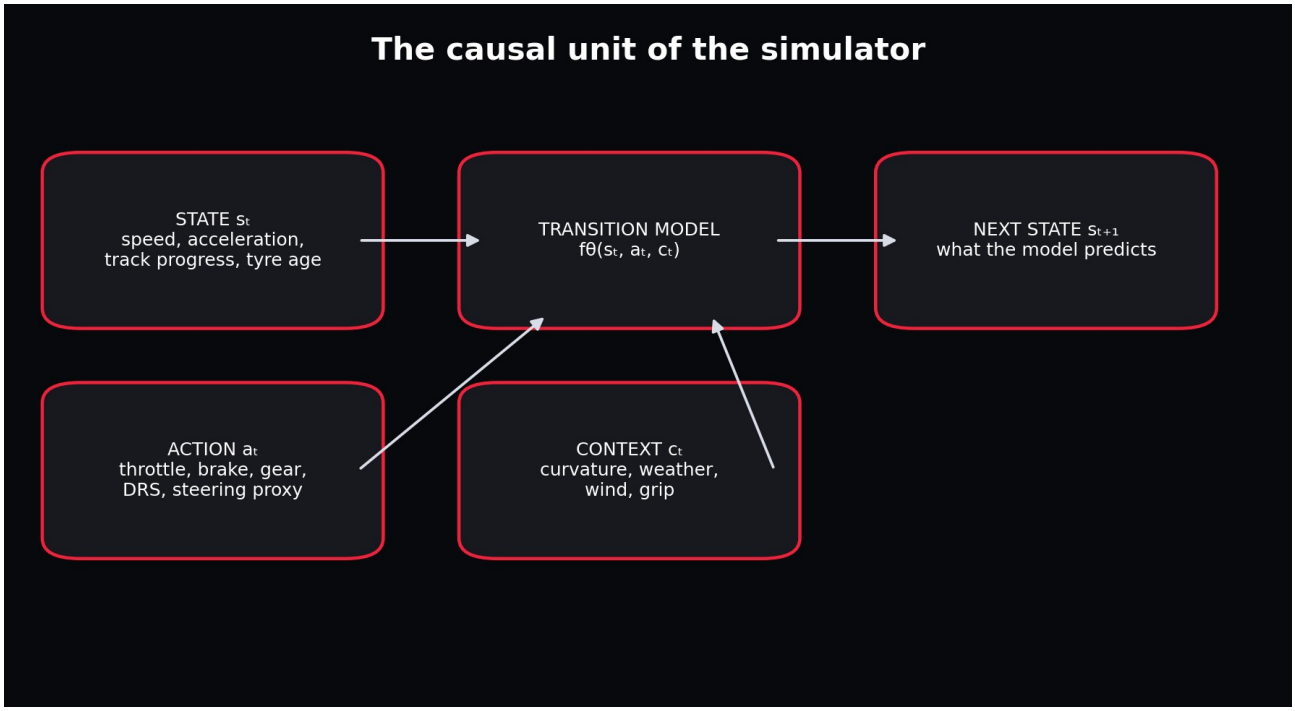
> **WHAT YOU ARE LEARNING:** Self-attention, causal masks, positional information and computational trade-offs.
>
> **WHY IT MATTERS:** Transformers are a useful comparator, not the automatic default.
>
> **YOU ARE FINISHED WHEN:** You can identify when attention adds value and when it adds avoidable cost.

Self-attention lets every step compute a weighted combination of other steps. This is powerful when distant events interact in content-dependent ways. Standard attention costs roughly quadratically with sequence length, while recurrent GRU/SSM inference keeps a fixed state. For an 8-second, 5 Hz V1 window, a Transformer is feasible; it is simply not necessary before the data and evaluation system are trusted.

# Part III - World Models and Dreamer

# Chapter 19 - What Makes a Model a World Model?

> **WHAT YOU ARE LEARNING:** Learned dynamics, observations, actions, rewards and imagination.
>
> **WHY IT MATTERS:** A sequence predictor becomes useful for planning only when it represents action-conditioned consequences.
>
> **YOU ARE FINISHED WHEN:** You can distinguish an unconditional forecast from a controllable learned world.




*The project models the conditional transition from present state and causes to next state.*

A world model is a learned representation of environment dynamics. It may predict observations directly or in a latent space. For control, it must respond to actions. Dreamer adds reward and continuation predictions, then trains an actor and critic inside imagined latent trajectories. Project APEX V1 stops before actor-critic control and concentrates on validating dynamics.

# Chapter 20 - Observation, State, Action, Context, Reward and Continuation

**WHAT YOU ARE LEARNING:** The semantic roles in a control-oriented model.

**WHY IT MATTERS:** Mixing action and state makes interventions invalid.

**YOU ARE FINISHED WHEN:** You can classify every canonical feature and defend ambiguous cases.

| Role | APEX V1 examples | Future F1 25 examples |
|---|---|---|
| Observation/state target | speed, acceleration, cyclic progress, tyre age | wheel speeds, temperatures, suspension, yaw, damage |
| Action | throttle, brake, gear, DRS, steering proxy | actual steering, clutch, overtake/ERS controls |
| Context | curvature, weather, wind, grip | setup, surface type, session state |

| | | |
|---|---|---|
| Reward/cost | not trained in V1 | lap-time progress, penalties, tyre/energy costs |
| Continuation | session/window availability | terminal crash, retirement, session end |

> **AMBIGUITY:** Gear can be viewed as an action or a state depending on whether the controller chooses it. The contract must state the intended causal interpretation.

# Chapter 21 - Latent Models and Autoencoders

> **WHAT YOU ARE LEARNING:** Encoders, latent representations and decoders.
>
> **WHY IT MATTERS:** High-dimensional observations can be compressed into dynamics-relevant state.
>
> **YOU ARE FINISHED WHEN:** You can explain why reconstruction alone may preserve visually dominant but control-irrelevant detail.

An encoder maps observation o_t to latent z_t. A decoder reconstructs or predicts observations from z_t. In low-dimensional telemetry, an encoder is optional because the input is already compact. In F1 25, a latent encoder becomes more useful as wheel-level, motion, setup, damage and multi-car features expand.

# Chapter 22 - Variational Autoencoders and Stochastic Latents

> **WHAT YOU ARE LEARNING:** Distributions, reparameterization and KL divergence.
>
> **WHY IT MATTERS:** An RSSM needs a trainable stochastic latent, not a random-number black box.
>
> **YOU ARE FINISHED WHEN:** You can explain the posterior, prior and why their KL is optimized.

The encoder outputs mean and standard deviation. A sample is written z = mean + std * epsilon, with epsilon drawn from a standard normal. This reparameterization lets gradients flow through mean and std. KL divergence penalizes disagreement between the observation-informed posterior and the predictive prior.

### Worked example

> **QUESTION:** What happens if KL weight is zero?

Step 1. The posterior can encode observations freely.

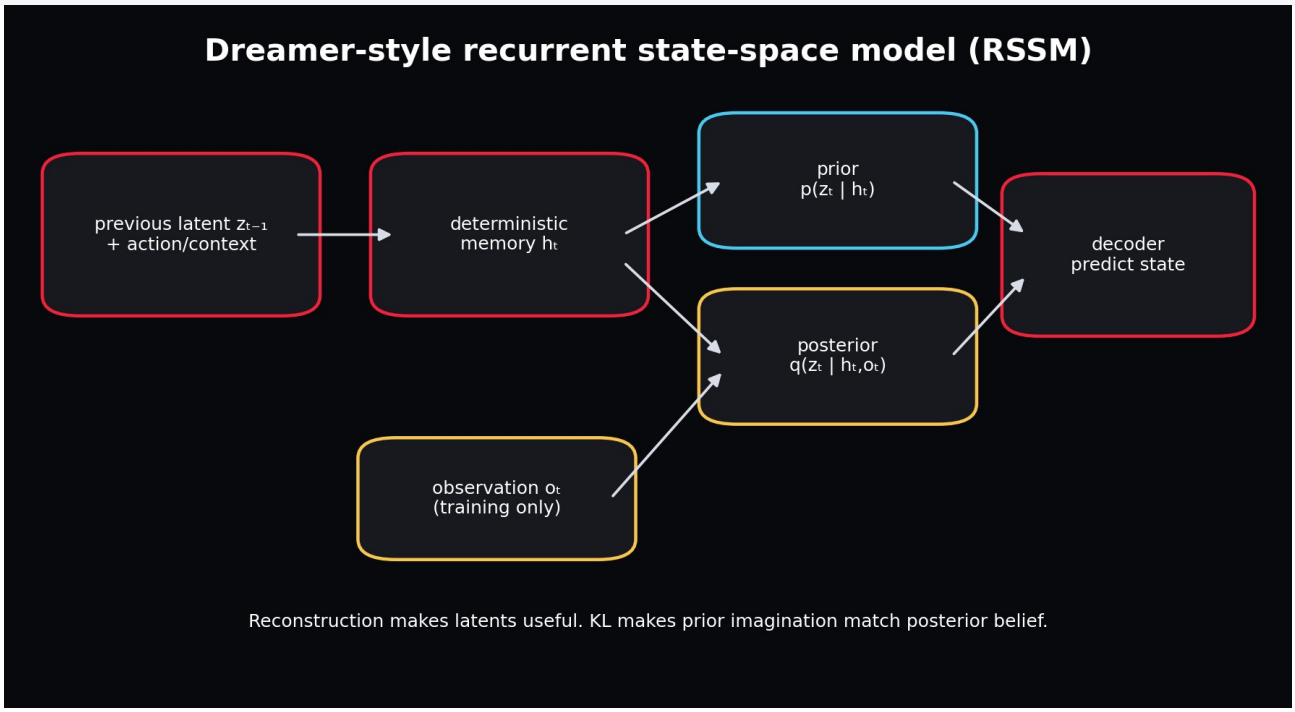
Step 2. The prior receives no pressure to match those latent states.

Step 3. At imagination time, only the prior is available, so rollouts can collapse.

> **ANSWER:** Reconstruction may look excellent during teacher-forced training while observation-free imagination fails.

# Chapter 23 - Recurrent State-Space Models (RSSMs)

> **WHAT YOU ARE LEARNING:** Deterministic recurrent memory plus stochastic latent state.
>
> **WHY IT MATTERS:** RSSMs combine persistent history with uncertainty and are central to Dreamer.
>
> **YOU ARE FINISHED WHEN:** You can trace posterior training and prior imagination step by step.




*During training the posterior sees the observation; during imagination the model samples from the prior.*

The deterministic state h_t stores long-lived context. The stochastic state z_t represents uncertainty or details that can branch. The transition first updates h_t from the previous latent and current

action/context. It then predicts a prior distribution over z_t. During training, a posterior also sees the observed state. The decoder predicts telemetry from h_t and z_t.

```
h = GRUCell(concat(z_previous, action_context), h_previous)
prior = p(z | h)
posterior = q(z | h, observed_state)   # training only
z = sample(posterior)                 # training
prediction = decoder(concat(h, z))
```

# Chapter 24 - Dreamer: Learning and Acting in Imagination

**WHAT YOU ARE LEARNING:** World-model learning, imagined trajectories, actor, critic and returns.

**WHY IT MATTERS:** This is the long-term conceptual target for an F1 25 agent.

**YOU ARE FINISHED WHEN:** You can explain why a good dynamics model must precede imagined policy optimization.

Dreamer learns the world model from experience, starts imagined trajectories from encoded states, trains a critic to estimate return, and trains an actor to choose actions that improve imagined return. The policy does not need to render every future observation; it acts in compact latent space. DreamerV3 introduced robust normalization and transformations that work across many domains with a common configuration.

**CRITICAL SEQUENCING:** Do not add actor-critic optimization to a dynamics model whose rollouts violate speed, position or intervention logic. The actor will exploit model errors.

# Chapter 25 - Rewards for an F1 Simulation Agent

**WHAT YOU ARE LEARNING:** Progress, lap time, track limits, tyre wear, energy, damage and multi-objective costs.

**WHY IT MATTERS:** A reward defines what the future controller will optimize.

**YOU ARE FINISHED WHEN:** You can identify reward hacking and propose guardrails.

A naive reward of speed encourages driving straight through corners. Progress per time is better but may still cut the track. A realistic objective combines progress, track validity, collision/damage penalties, tyre and energy costs, and terminal outcomes. Reward components should be logged separately so an agent cannot improve the total by exploiting one hidden term.

| Reward component | Desired behaviour | Possible exploit |
|---|---|---|
| Track progress | Complete distance quickly | Cut track or teleport if state errors permit |
| Lap time | Fast complete laps | Ignore tyre or fuel sustainability |
| Track-limit penalty | Stay within circuit | Drive unrealistically slowly |
| Damage penalty | Preserve car | Avoid racing or overtaking |
| Tyre/energy cost | Manage resources | Sacrifice too much pace |

# Chapter 26 - Planning: MPC and CEM Before Full RL

**WHAT YOU ARE LEARNING:** Model predictive control, candidate action sequences and cross-entropy method search.

**WHY IT MATTERS:** Planning can use a learned world model without immediately training an actor.

**YOU ARE FINISHED WHEN:** You can describe receding-horizon control and its compute trade-off.

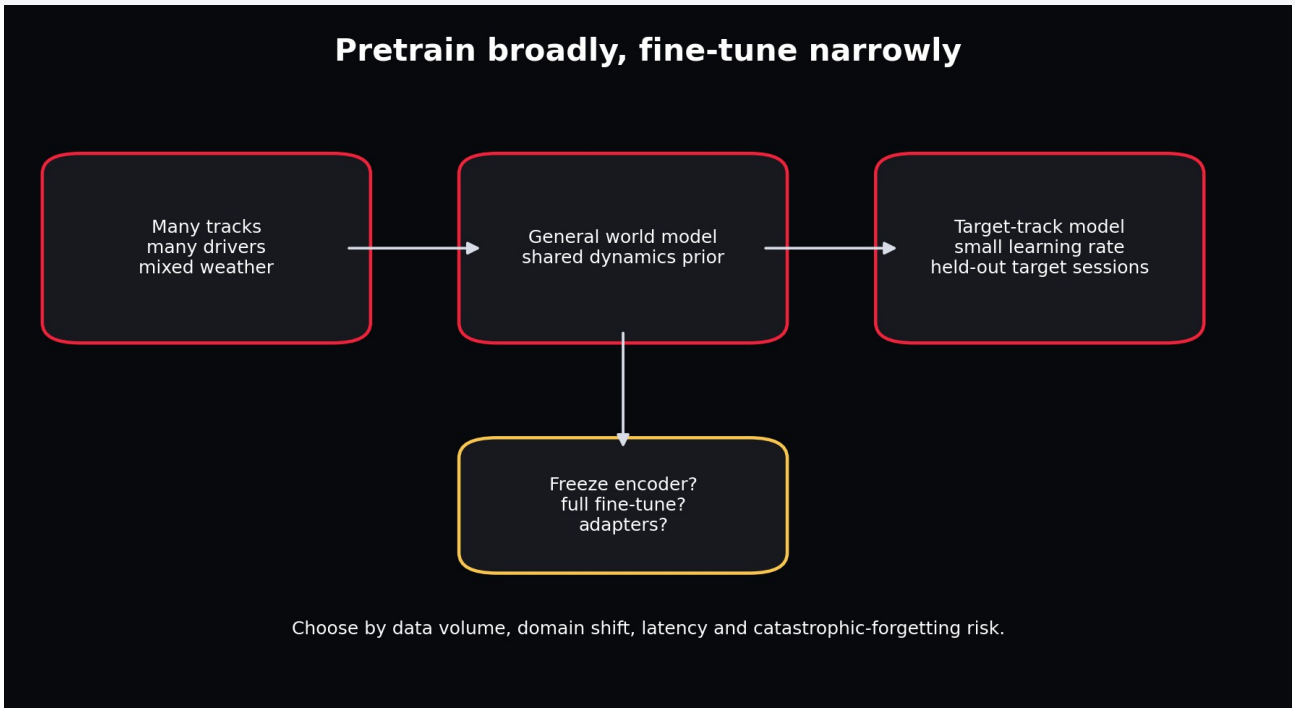
MPC repeatedly samples candidate action sequences, rolls each through the world model, scores predicted outcomes, executes only the first action, then replans from the new real observation. CEM iteratively refits the candidate distribution to high-scoring sequences. This is easier to audit than a learned actor but can be expensive at high frequency.

# Chapter 27 - Fine-Tuning Existing Models Versus Building Your Own

**WHAT YOU ARE LEARNING:** Domain adaptation paths for world models and pretrained time-series backbones.

**WHY IT MATTERS:** World models are not as plug-and-play as language models because observation/action contracts differ.

**YOU ARE FINISHED WHEN:** You can choose among warm-starting, frozen features, adapters and full fine-tuning.




*General pretraining provides a dynamics prior; target-track fine-tuning adapts with limited data.*

There are three realistic reuse paths. First, pretrain the APEX architecture across many tracks and fine-tune on a target track or driver. Second, use a pretrained time-series backbone such as PatchTST or Chronos for representations/forecasting, then add action-conditioned heads; this is useful but not automatically a control world model. Third, adapt an existing Dreamer implementation, which requires defining observation/action spaces, replay data, reward and continuation heads, and often rewriting environment interfaces.

| Path | Strength | Main risk |
|---|---|---|
| APEX cross-track pretraining | Exact contract and causal inputs | Needs enough diverse F1 data |
| Frozen time-series foundation features | Fast low-data benchmark | Backbone may ignore actions or domain physics |
| Full/flexible fine-tune | Maximum adaptation | Overfitting and catastrophic forgetting |
| Dreamer codebase adaptation | Complete control algorithm | High complexity; environment mismatch; difficult debugging |

# Part IV - Project Scope and Architecture

## Chapter 28 - V1 Product Definition

**WHAT YOU ARE LEARNING:** A measurable first release and explicit non-goals.

**WHY IT MATTERS:** A world-model project can expand without limit unless the first operational contract is fixed.

**YOU ARE FINISHED WHEN:** You can state V1 inputs, outputs, horizon, users and acceptance tests.

V1 takes a history of canonical telemetry plus a proposed future action/context sequence and predicts speed, acceleration, cyclic track progress and tyre age over a short horizon. It supports historical exploration, model comparison, intervention sliders and artifact inspection through a UI. It does not optimize driving actions or claim race-strategy accuracy.

| Dimension | V1 decision |
|---|---|
| Prediction unit | One driver in one session window |
| History | 32 frames at 5 Hz = 6.4 seconds |
| Default horizon | 8 frames = 1.6 seconds |
| Data | Offline synthetic plus optional FastF1/OpenF1 |
| Primary model | Deterministic GRU |
| Challengers | Selective SSM and RSSM |
| Primary metric | Speed MAE plus horizon curve |
| Guardrails | Physical violation checks and session holdout |
| Interface | Gradio UI and FastAPI inspection service |

## Chapter 29 - Data Limitations Before F1 25

**WHAT YOU ARE LEARNING:** What public historical sources contain and what they do not.

**WHY IT MATTERS:** The model cannot infer reliable causal control dynamics from signals that do not exist.

**YOU ARE FINISHED WHEN:** You can distinguish measured action, proxy, context and unavailable variable.

FastF1 exposes timing, car telemetry, position, tyre and weather information through convenient Pandas-based objects. OpenF1 exposes historical JSON/CSV endpoints for car data, location, laps, weather, stints and more. Public data lacks the complete high-rate player control, wheel and setup state available in the game UDP feed. Steering is therefore approximated from heading change in V1.

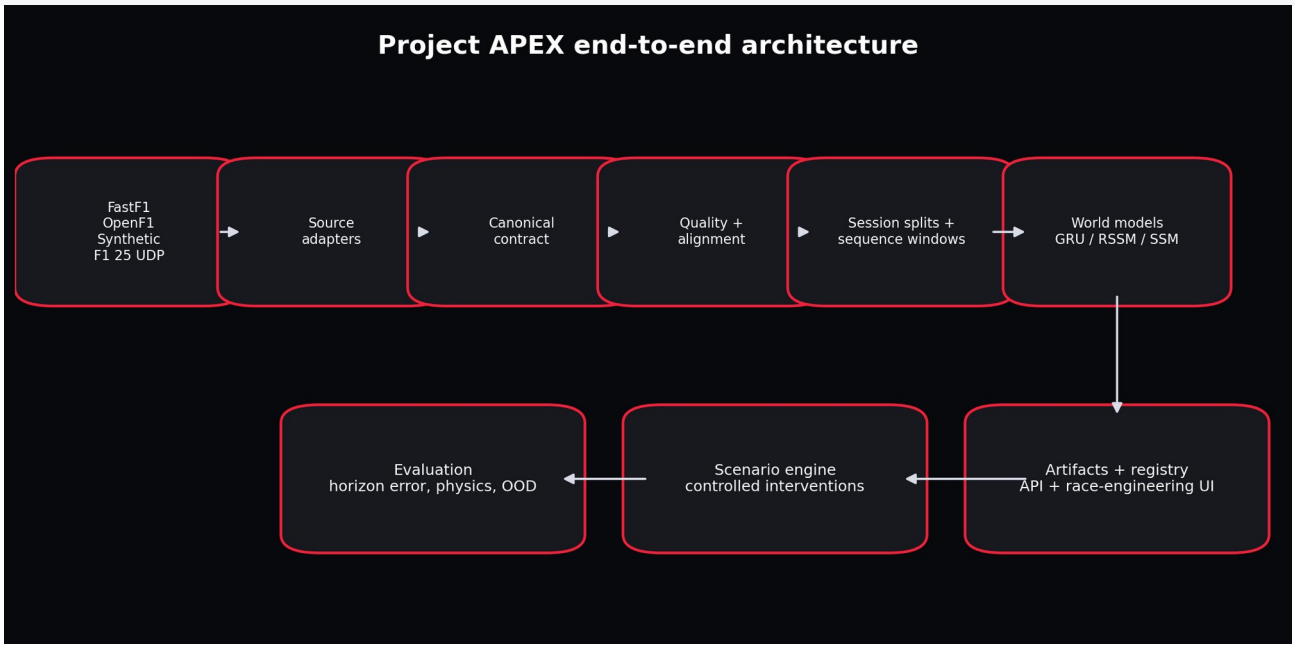
**CAUSAL CAUTION:** A steering proxy derived from future path geometry is not equivalent to a driver steering command. Treat it as context/derived action and document the limitation.

# Chapter 30 - Full System Architecture

**WHAT YOU ARE LEARNING:** Every component from source data to user interface.

**WHY IT MATTERS:** A clear architecture makes the project replaceable and testable instead of becoming one notebook.

**YOU ARE FINISHED WHEN:** You can trace a frame from source endpoint to UI chart and name every persisted artifact.




*The full system separates unstable external sources from a stable internal contract and reusable modelling core.*

The source adapter is the anti-corruption layer. The canonical contract is the stable boundary. Quality and alignment stages protect downstream assumptions. Session splits and train-only

normalization protect evaluation. Model modules share a common history/future contract. Evaluation, simulation and publication consume model artifacts. The registry and interfaces never need to know whether the source was synthetic, FastF1 or F1 25.

# Chapter 31 - Repository Architecture

**WHAT YOU ARE LEARNING:** How folders mirror responsibilities.

**WHY IT MATTERS:** Folder structure should reveal system boundaries before code is opened.

**YOU ARE FINISHED WHEN:** You can find any behaviour by following the responsibility map.

```
src/apexsim/
  data/          # adapters, validation, features, windows
  models/        # GRU, RSSM, selective SSM, baselines
  pipeline/      # idempotent stages and orchestration
  config.py      # validated experiment intent
  contracts.py   # source-independent schema
  training.py    # parameter optimization
  evaluation.py  # horizon and physical metrics
  simulation.py  # scenario interventions and rollout
  registry.py    # durable run state
  serving.py     # REST inspection interface
  ui.py          # interactive client
```

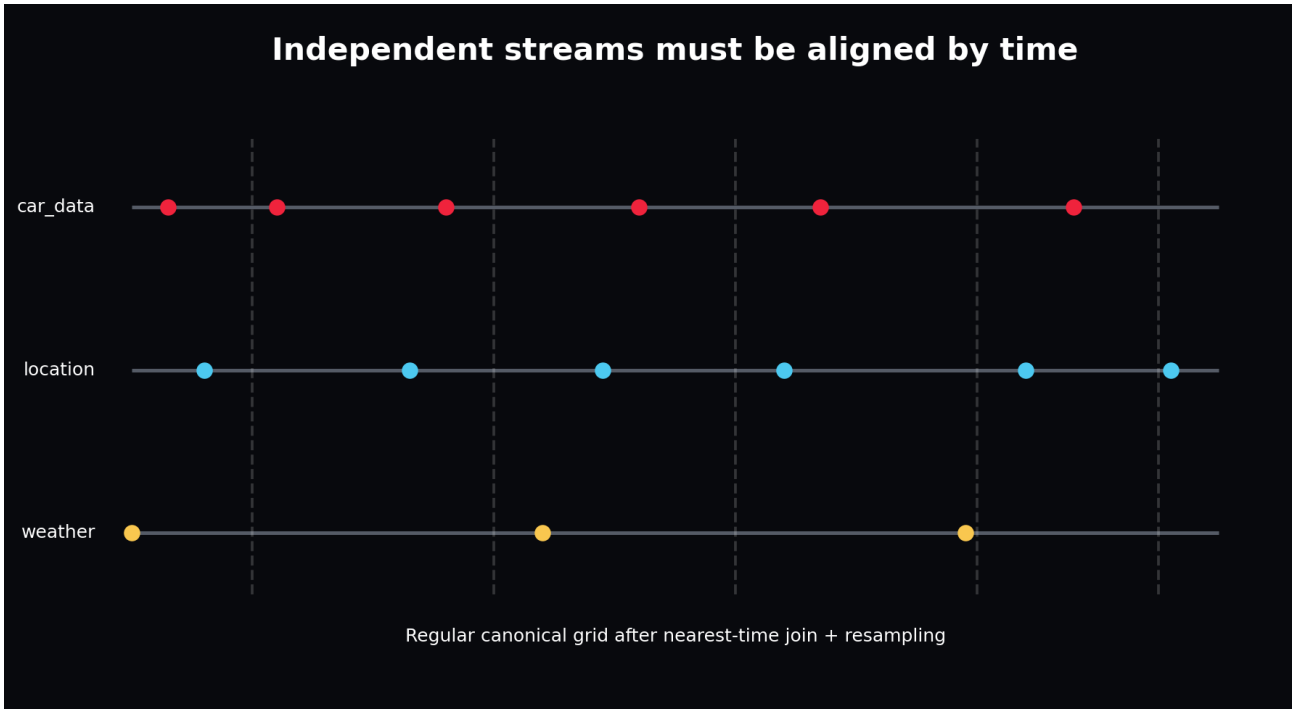
**DESIGN TEST:** If replacing Gradio with React requires editing the model or ingestion code, the boundary is wrong.

# Chapter 32 - Data Flow From API to Tensor

**WHAT YOU ARE LEARNING:** Raw payload, normalized table, validated frame, episode split, window and batch.

**WHY IT MATTERS:** Every transformation can introduce leakage or semantic corruption.

**YOU ARE FINISHED WHEN:** You can describe the representation and invariants at each stage.




*Asynchronous source events become a regular canonical time grid only after timestamp-aware alignment.*

| Stage | Representation | Key invariant |
|---|---|---|
| Raw landing | Original JSON/cache/dataframe | Preserve source timestamps and identifiers |
| Adapter output | Canonical dataframe | Units and column meanings are stable |
| Validated data | Admitted dataframe + report | Ranges, keys and missingness meet contract |
| Split data | Session ID lists | No session appears in multiple sets |
| Window dataset | History/future tensors | No window crosses an episode |
| Batch | [B,T,F] tensors | Axis semantics and normalization match model |

# Chapter 33 - Decision Records: Why This Implementation?

**WHAT YOU ARE LEARNING:** Architecture decision records and reversible choices.

**WHY IT MATTERS:** The project should teach how to choose, not only what was chosen.

**YOU ARE FINISHED WHEN:** You can write a one-page decision with context, options, evidence, consequence and revisit trigger.

| Decision | Chosen now | Alternative | Revisit when |
|---|---|---|---|
| UI | Gradio | React/Next.js | Multiple users, auth, richer client state |
| V1 dynamics | GRU | Transformer/Mamba | Long histories or attention bottleneck measured |
| Storage | CSV + JSON artifacts | Parquet/object store | Data scale or query cost increases |
| Registry | SQLite | Postgres/MLflow | Concurrent teams and remote deployment |
| Orchestration | Local runner + thin Airflow DAG | Distributed workflow system | Scheduled multi-session scale requires it |

# Part V - Ingestion, Contracts and Data Flow

# Chapter 34 - FastF1 Ingestion

**WHAT YOU ARE LEARNING:** Session loading, caching, laps, telemetry and conversion.

**WHY IT MATTERS:** FastF1 is the most convenient offline historical starting point.

**YOU ARE FINISHED WHEN:** You can ingest one driver session and identify where source-specific assumptions end.

```
session = fastf1.get_session(year, event, session_code)
session.load(telemetry=True, laps=True, weather=True)
driver_laps = session.laps.pick_drivers(driver)
for _, lap in driver_laps.iterlaps():
    telemetry = lap.get_telemetry()
```

FastF1 provides extended Pandas objects and caching. The adapter resamples each lap, converts speed to m/s, estimates acceleration, creates cyclic progress, maps controls to fractions, and writes the canonical schema. Keep the raw cache because future adapter corrections should not require another external download.

**VERSION AWARENESS:** External APIs and library fields can change. Pin and record dependency versions, preserve raw source data, and validate every adapter run.

# Chapter 35 - OpenF1 Ingestion

> **WHAT YOU ARE LEARNING:** HTTP endpoint queries, JSON dataframes and asynchronous stream joins.
>
> **WHY IT MATTERS:** OpenF1 provides broad historical endpoints but does not guarantee row-by-row alignment across them.
>
> **YOU ARE FINISHED WHEN:** You can merge car, location and weather streams by time with an explicit tolerance.

OpenF1 historical data begins in 2023 and is accessible without authentication, while real-time access has separate requirements. Car data is sampled at only a few hertz, and other endpoints have their own cadence. The adapter parses UTC timestamps, sorts each stream, uses merge_asof with tolerances, then resamples onto a regular grid.

```
merged = pd.merge_asof(
    car.sort_values("date"),
    location.sort_values("date"),
    on="date",
    direction="nearest",
    tolerance=pd.Timedelta("1s"),
)
```

### Worked example

> **QUESTION:** Why is concatenating the nth car_data row with the nth location row invalid?

Step 1. The endpoints can start at different times and sample at different rates.

Step 2. Dropped or delayed records shift row indices.

Step 3. The resulting row may combine speed from one instant with position from another.

> **ANSWER:** Join by timestamp with a documented tolerance, inspect unmatched rates, and quarantine sessions with excessive alignment gaps.

### Run either adapter through the same downstream pipeline

```
apexsim ingest-fastf1 --year 2025 --event Monza --session R --driver VER   --output data/raw/monza_ver.csv

apexsim run-canonical --input-path data/raw/monza_ver.csv   --config configs/fast.yaml --run-id monza_ver_gru
```

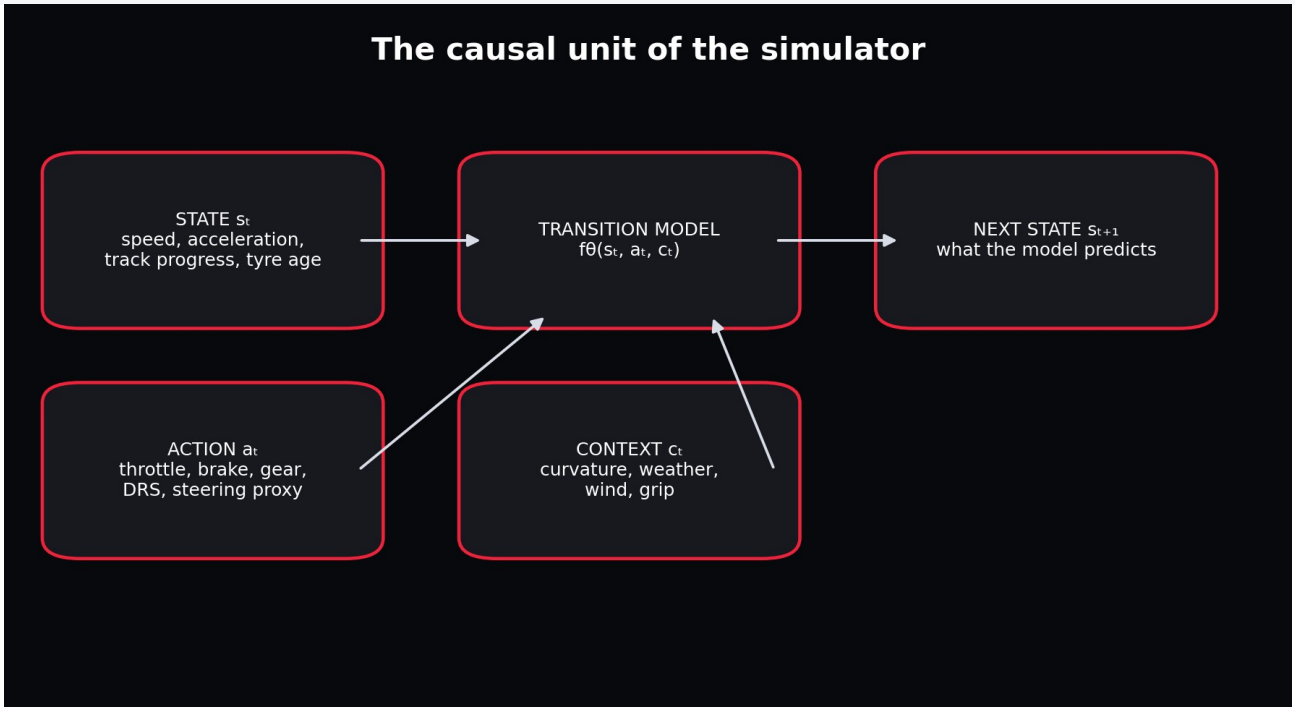
The adapter owns source-specific retrieval, units and alignment. `run-canonical` copies the exact adapter output into the immutable run directory, then uses the identical validation, session splitting, train-only normalization, training, evaluation, ablation and publication stages used by synthetic data. This prevents the real-data path from becoming a second untested application.

# Chapter 36 - The Canonical Telemetry Contract

**WHAT YOU ARE LEARNING:** Stable identity, state, action, context and auxiliary columns.

**WHY IT MATTERS:** The contract lets the entire downstream system ignore source-specific naming.

**YOU ARE FINISHED WHEN:** You can read one row as a causal sentence.




*The contract preserves causal roles rather than presenting one undifferentiated feature list.*

| Group | Columns |
|---|---|
| Identity | session_id, source, track_id, driver_id, timestamp_s, lap_number |
| State targets | speed_mps, acceleration_mps2, track_progress_sin/cos, tyre_age_laps |
| Actions | throttle, brake, gear_norm, drs, steering_proxy |
| Context | curvature, air_temp_c, track_temp_c, rainfall, wind_speed_mps, grip_level |
| Auxiliary | lap_distance_m, x_m, y_m, rpm, gear, compound_id, is_pit, safety_car |

**CONTRACT RULE:** Adding a column does not make it causally valid. State whether it is observable at inference time and whether the user/controller can intervene on it.

# Chapter 37 - Validation and Quality Gates

**WHAT YOU ARE LEARNING:** Schema, nulls, duplicate frame keys, ranges, gaps and warnings.

**WHY IT MATTERS:** Bad data should fail near ingestion, not become mysterious model error.

**YOU ARE FINISHED WHEN:** You can classify each condition as error, warning, quarantine or monitor.

```
report = validate_canonical_frame(frame, strict=True)
assert report.null_cells == 0
assert report.duplicate_frames == 0
assert report.out_of_range_cells == 0
```

| Condition | Default treatment | Reason |
|---|---|---|
| Missing required column | Error | Downstream semantics undefined |
| Throttle = 1.5 | Error | Impossible under unit-fraction contract |
| Unusually wet session | Warning/monitor | Valid but distribution-shifting |
| Long timestamp gap | Split/quarantine | Interpolation would invent trajectory |
| Rare compound | Keep + monitor | Operationally important tail case |
| Duplicate session/driver/time key | Error | Ambiguous truth |

**INSTRUCTOR CHECKPOINT:** Deliberately corrupt one row, predict which report count changes, then run the test.

# Chapter 38 - Resampling and Temporal Alignment

**WHAT YOU ARE LEARNING:** Regular grids, interpolation policies, forward fill and gap handling.

**WHY IT MATTERS:** Sequence models expect consistent temporal meaning.

**YOU ARE FINISHED WHEN:** You can choose interpolation by variable type and refuse unsafe gaps.

| Variable type | Example | Reasonable policy |
|---|---|---|
| Continuous sensor | speed, temperature | Linear interpolation over short gaps |
| Piecewise control | gear, DRS, safety car | Forward fill or nearest |
| Event | pit entry, penalty | Preserve timestamp; derive active state carefully |
| Circular angle | heading | Unwrap, interpolate, wrap |
| Large gap | missing 5 seconds | Split episode; do not interpolate |

Resampling does not create information. It creates a regular representation of available information under explicit assumptions. Store source timestamps and alignment diagnostics so later analysis can distinguish model error from synchronization error.

# Chapter 39 - Feature Engineering

**WHAT YOU ARE LEARNING:** Derived acceleration, steering proxy, curvature, cyclic progress and normalized controls.

**WHY IT MATTERS:** Useful inductive bias can reduce data requirements but can also leak future information.

**YOU ARE FINISHED WHEN:** You can audit each feature for causality and inference availability.

Curvature can be derived from static track geometry and is valid future context. Steering proxy derived from heading changes is more ambiguous: when computed from the future trajectory it partially encodes what the car actually did. In V1 it is a documented approximation; in F1 25 it should be replaced by actual steering control.

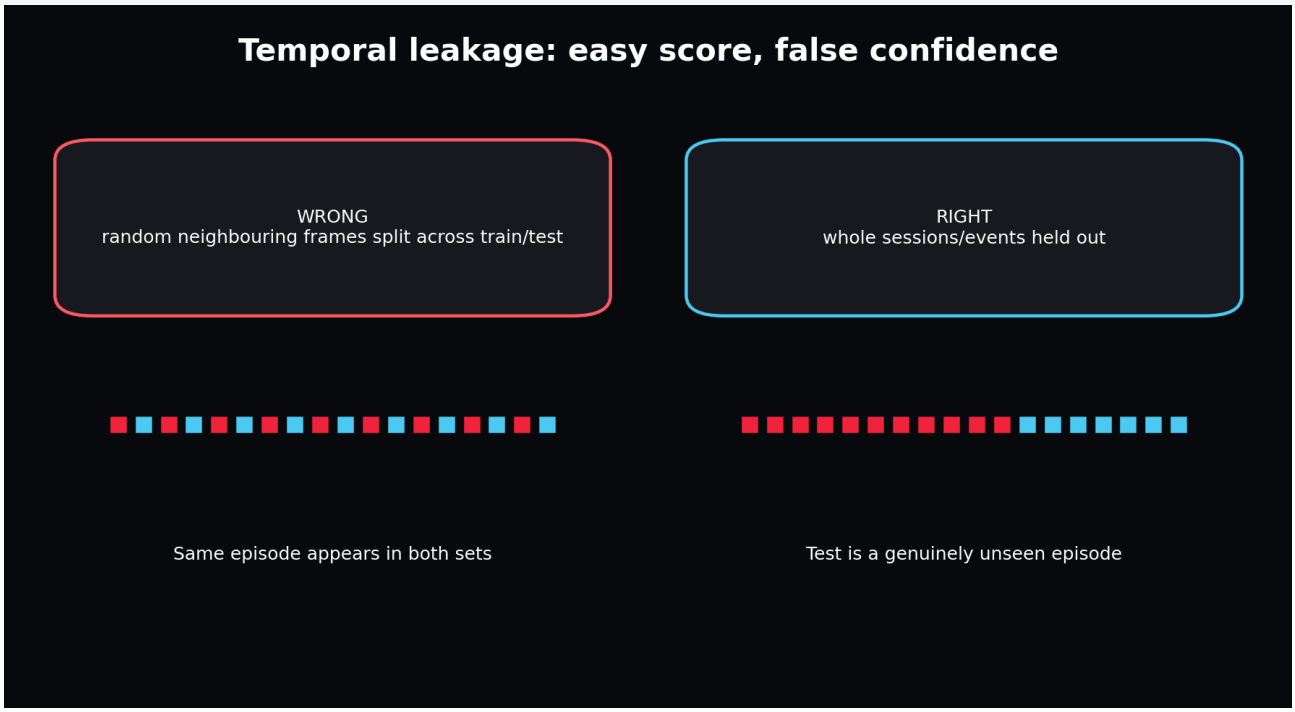
**LEAKAGE AUDIT:** For each feature, ask: could this exact value be known at the moment the prediction starts? If not, it belongs in the target, an oracle experiment, or nowhere.

# Chapter 40 - Session-Level Splitting

**WHAT YOU ARE LEARNING:** Train, validation and test partitions by independent episode.

**WHY IT MATTERS:** Evaluation must measure generalization rather than memorization of neighbouring frames.

**YOU ARE FINISHED WHEN:** You can implement deterministic splits and prove disjointness.




*Session holdouts are harder and more honest than random frame splits.*

```
assert set(splits["train"]).isdisjoint(splits["val"])
assert set(splits["train"]).isdisjoint(splits["test"])
assert set(splits["val"]).isdisjoint(splits["test"])
```

For stronger evaluation, hold out complete tracks, drivers, weather regimes or seasons. Validation should resemble the deployment question. A random session holdout does not prove generalization to a new circuit.

# Chapter 41 - Train-Only Standardization

**WHAT YOU ARE LEARNING:** Fitting and persisting normalization statistics.

**WHY IT MATTERS:** A model checkpoint is unusable without the exact feature order and scaling transform.

**YOU ARE FINISHED WHEN:** You can serialize and invert predictions correctly.

```
standardizer = Standardizer.fit(train_frame)
x_z = standardizer.transform_inputs(x_raw)
y_z = standardizer.transform_targets(y_raw)
y_raw_again = standardizer.inverse_targets(y_z)
```

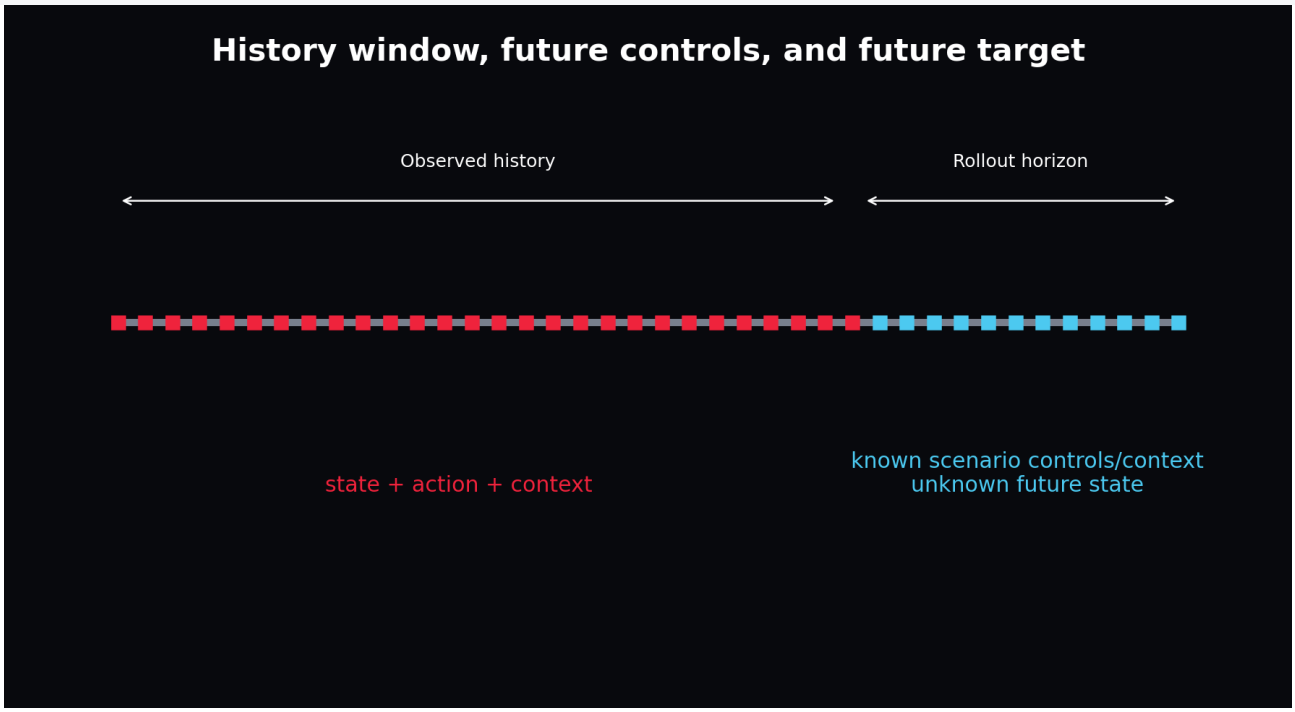
**PRODUCTION ARTIFACT:** The standardizer is part of the model. Version it with feature order, units and checkpoint.

# Chapter 42 - Sequence Windows and DataLoader

**WHAT YOU ARE LEARNING:** History, known future causes, future target, episode safety and batching.

**WHY IT MATTERS:** Window semantics determine what the model is allowed to know.

**YOU ARE FINISHED WHEN:** You can trace one manifest/index into tensors and verify all shapes.




*The model observes history, receives future scenario causes, and predicts future states.*

```
    return {
        "history": [T_history, F_input],
        "future_inputs": [T_future, F_input],
        "future_targets": [T_future, F_state],
        "session_id": session_id,
    }
```

The future input tensor includes placeholders for state because the common feature order is convenient. During autoregressive rollout, the model overwrites the state portion with its own previous prediction while preserving the planned action/context portion.

# Part VI - Guided Construction of V1

# Chapter 43 - Environment Setup and First Proof

**WHAT YOU ARE LEARNING:** Install, test, compile and run the fast configuration.

**WHY IT MATTERS:** You need a known-good reference before changing anything.

**YOU ARE FINISHED WHEN:** Six tests pass and a named run produces a complete artifact tree.

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
pytest -q
python -m py_compile dags/apexsim_dag.py
apexsim run --config configs/fast.yaml --run-id my_first_run
```

**PREDICT BEFORE RUNNING:** Write the expected folders and files. After execution, explain why each exists.

# Chapter 44 - Generate and Inspect the Offline World

**WHAT YOU ARE LEARNING:** Track geometry, driver policy, weather, tyre wear and causal dynamics.

**WHY IT MATTERS:** Offline reproducibility lets you debug before external data or the game exists.

**YOU ARE FINISHED WHEN:** You can alter one physical factor and predict the directional change.

```
config = load_config("configs/fast.yaml")
frame = generate_synthetic_sessions(config, "data/raw/synthetic.csv")
print(frame.shape)
print(frame.groupby("session_id").speed_mps.max())
```

Open `data/synthetic.py`. Follow distance to progress, progress to geometry, curvature to target speed, driver error to throttle/brake, and forces to next speed. This is the first complete simulator in the project. The world model later learns an approximation of these hidden rules from outputs.

# Chapter 45 - Baseline Models

**WHAT YOU ARE LEARNING:** Persistence and linear one-step transitions.

**WHY IT MATTERS:** Deep learning earns its complexity by beating correct controls.

**YOU ARE FINISHED WHEN:** You can state when a baseline is stronger operationally despite lower peak accuracy.

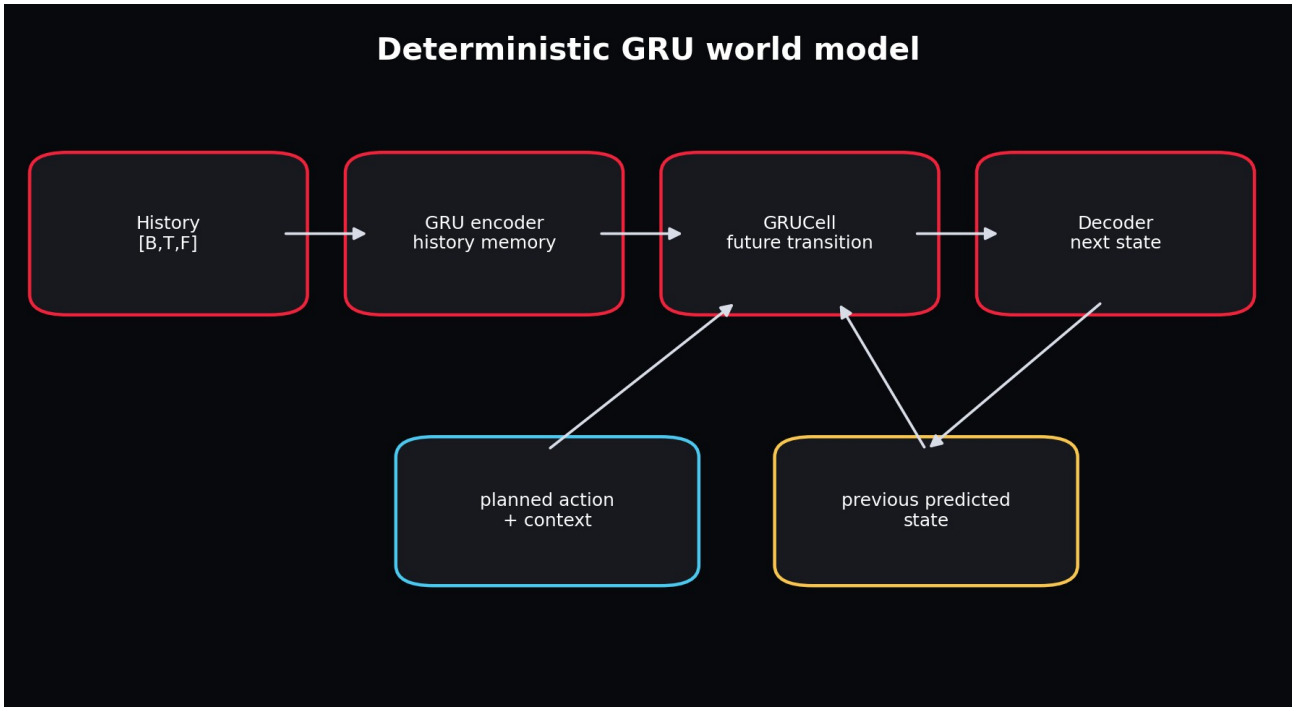
Persistence predicts no change. A ridge transition predicts next state from current features. Linear models train quickly, expose coefficient direction, and power cheap ablation probes. They are especially useful for detecting target alignment mistakes: a suspiciously perfect linear result may indicate leakage.

# Chapter 46 - Build the GRU World Model

**WHAT YOU ARE LEARNING:** History encoding, recurrent transition, decoding and autoregressive state replacement.

**WHY IT MATTERS:** This is the primary V1 implementation.

**YOU ARE FINISHED WHEN:** You can trace every tensor and explain why future targets never enter forward inference.




*GRU rollout reuses the predicted state at the next step, so errors compound honestly.*

```
_, hidden = self.encoder(history)
hidden_t = hidden[-1]
previous_state = history[:, -1, :state_dim]
for step in range(future_inputs.shape[1]):
    step_input = future_inputs[:, step].clone()
    step_input[:, :state_dim] = previous_state
    hidden_t = self.transition(step_input, hidden_t)
    previous_state = self.decoder(hidden_t)
    predictions.append(previous_state)
```

Line 1 compresses the observed past. Line 2 selects the top recurrent layer. Line 3 extracts the latest normalized state. The loop copies planned future features, replaces the state fields with model belief, updates memory, decodes a next state, and stores it. The model is therefore exposed to its own mistakes during every multi-step forward pass.

# Chapter 47 - Loss Functions for Telemetry

**WHAT YOU ARE LEARNING:** MSE, MAE, Smooth L1, per-channel weighting and geometric constraints.

**WHY IT MATTERS:** A scalar loss decides which errors the model prioritizes.

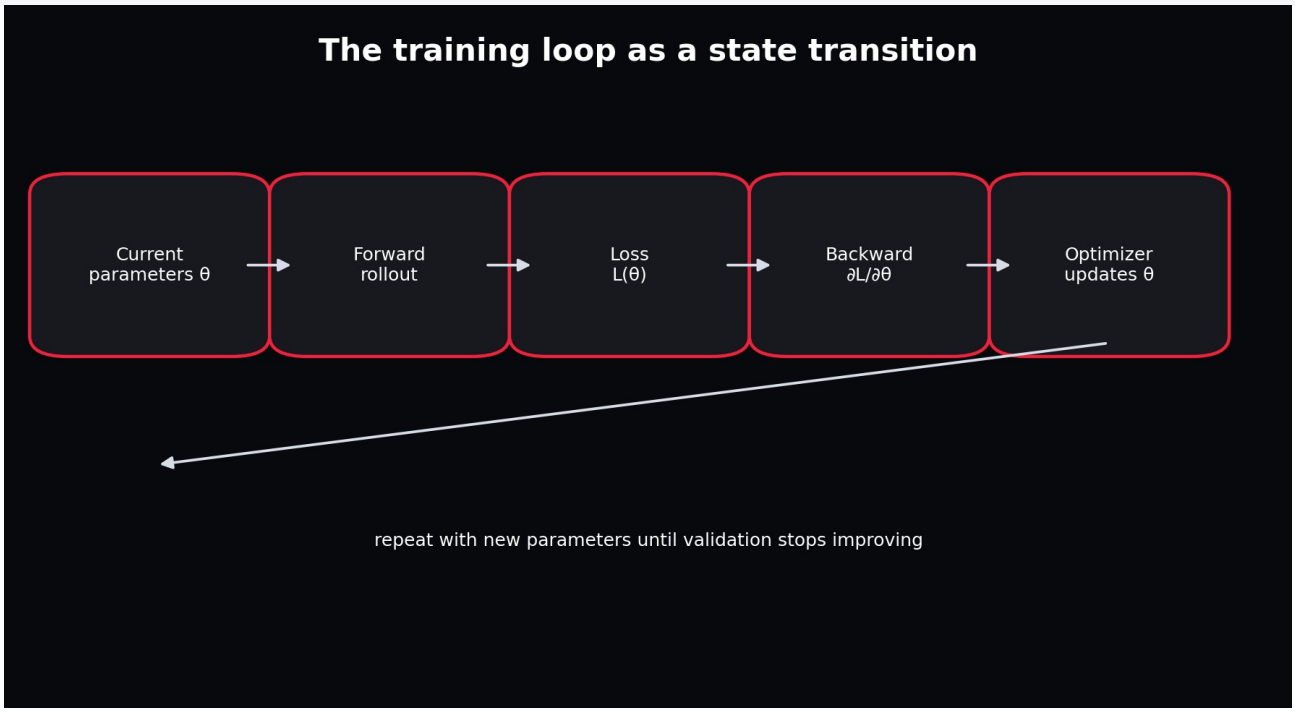
**YOU ARE FINISHED WHEN:** You can choose a loss based on outliers, units and operational cost.

Project APEX uses Smooth L1 in normalized space. Small errors behave approximately quadratically, encouraging precision; large errors behave linearly, reducing domination by outliers. Normalization prevents speed magnitude from completely overwhelming cyclic progress.

> **FUTURE IMPROVEMENT:** Add a unit-circle penalty for progress sine/cosine and a non-negative speed transform if physical violations become material.

# Chapter 48 - The Training Loop, Line by Line

> **WHAT YOU ARE LEARNING:** Batches, forward pass, loss, backward pass, clipping, update, validation and checkpoint.
>
> **WHY IT MATTERS:** Debugging requires knowing exactly when model state changes.
>
> **YOU ARE FINISHED WHEN:** You can annotate the loop without saying "PyTorch handles it".




*Each batch computes a local direction for changing the parameter state.*

```
for batch in train_loader:
    optimizer.zero_grad(set_to_none=True)
    loss, prediction = batch_loss(model, batch)
    loss.backward()
    clip_grad_norm_(model.parameters(), grad_clip)
    optimizer.step()
```

Validation runs with `model.eval()` and `torch.no_grad()`. Eval mode changes dropout/batch normalization behaviour. No-grad prevents graph construction. The best validation checkpoint is restored after training, rather than keeping the last epoch by habit.

# Chapter 49 - Overfit-One-Batch Debugging

**WHAT YOU ARE LEARNING:** A minimal training sanity test.

**WHY IT MATTERS:** Long training cannot repair a broken target, disconnected gradient or wrong axis.

**YOU ARE FINISHED WHEN:** A tiny dataset loss falls dramatically and predictions move toward targets.

1. Choose 16-64 windows from one session.
2. Disable regularization and augmentation.
3. Use a moderately large model.
4. Train for many updates on exactly those windows.
5. Inspect prediction and target arrays directly.
6. If loss does not collapse, debug code and data before collecting more data.

# Chapter 50 - Build the Selective SSM Challenger

**WHAT YOU ARE LEARNING:** Stable decay, input projection, write gate and stacked recurrent state.

**WHY IT MATTERS:** The challenger teaches long-memory design without hidden dependencies.

**YOU ARE FINISHED WHEN:** You can explain which part corresponds to retention and which part writes new information.

```
decay = sigmoid(logit_decay)
candidate = tanh(input_projection(x))
gate = sigmoid(gate_projection(x))
state = decay * state + (1 - decay) * gate * candidate
```

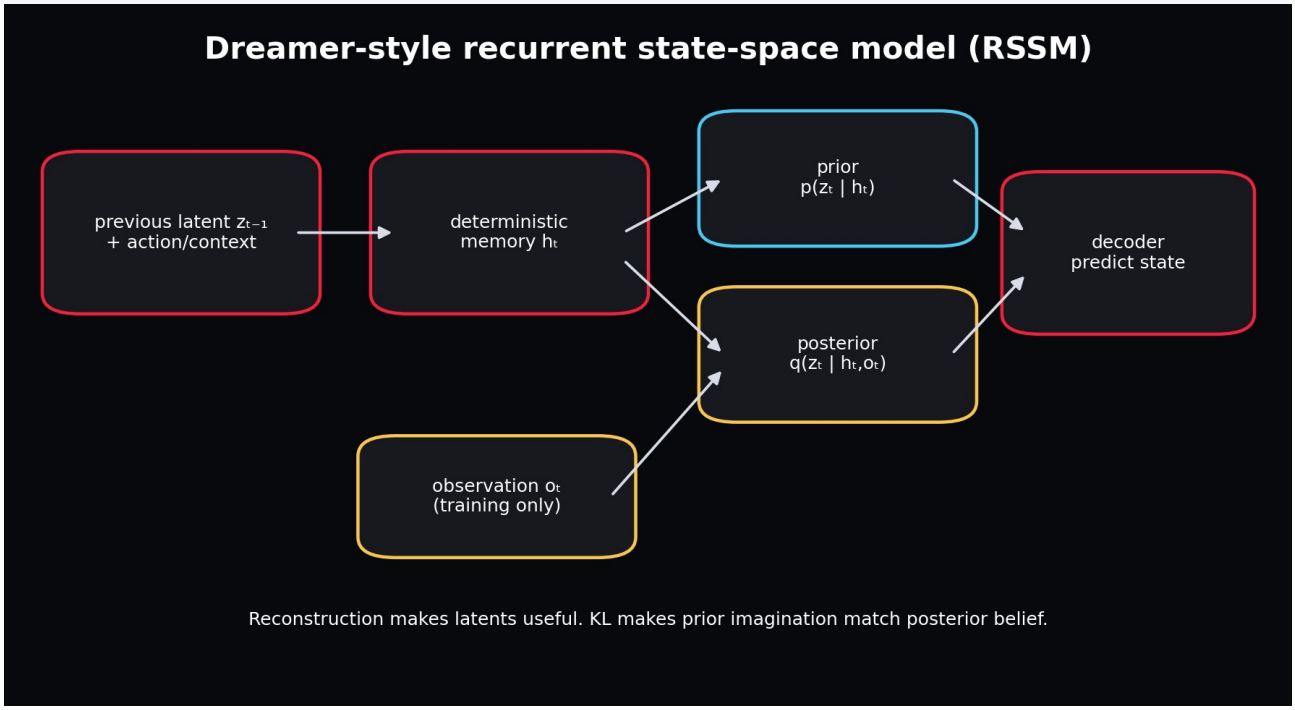
Decay near one retains memory. The gate controls input-dependent writing. Multiple cells stack state transformations. A full Mamba layer adds a more sophisticated parameterization, local convolution/gating components and highly optimized selective scan. Use the simple cell to validate the need for SSM behaviour before adding that complexity.

# Chapter 51 - Build the Dreamer-Style RSSM

**WHAT YOU ARE LEARNING:** Deterministic memory, prior, posterior, sampling, decoder and KL.

**WHY IT MATTERS:** This is the conceptual bridge to future Dreamer control.

**YOU ARE FINISHED WHEN:** You can run training and imagination modes and explain their difference.




*Posterior training and prior imagination must converge toward compatible latent states.*

```
h = gru(concat(z, action_context), h)
prior = distribution(prior_head(h))
posterior = distribution(posterior_head(concat(h, target_state)))
z = posterior.rsample()
kl = KL(posterior || prior)
prediction = decoder(concat(h, z))
```

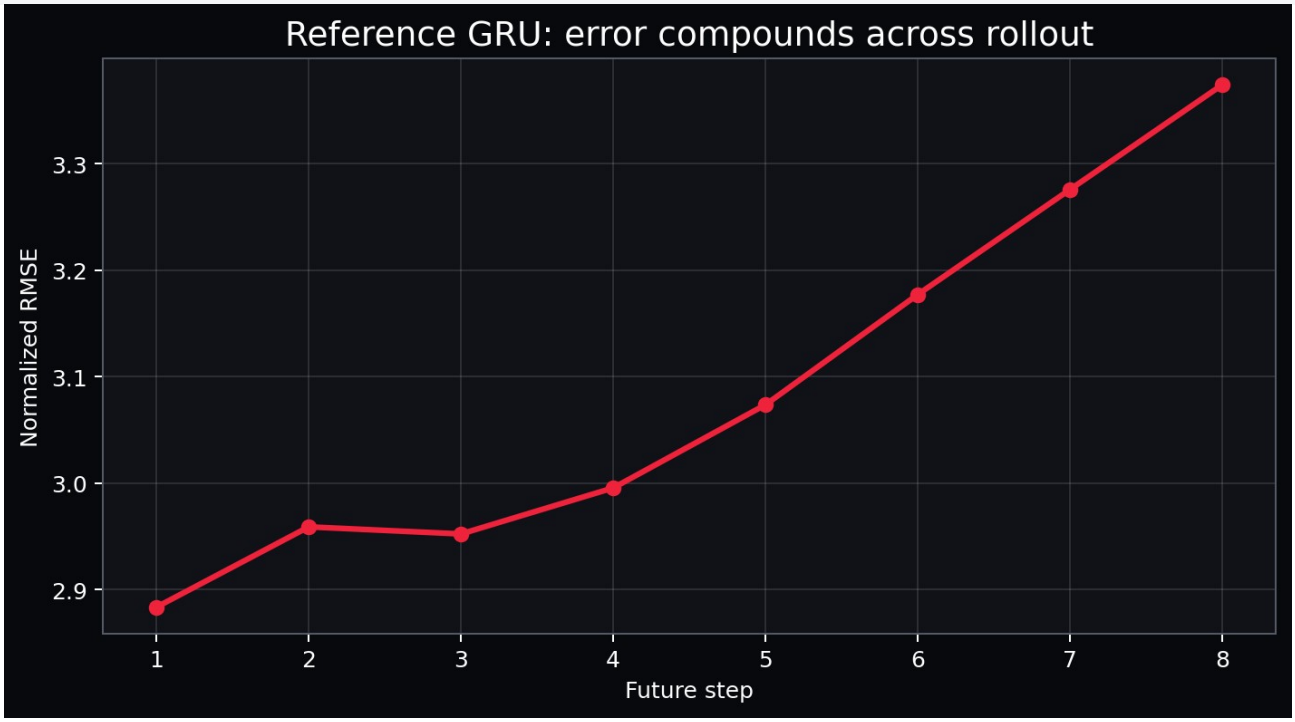
**REFERENCE RESULT:** Under the intentionally tiny fast budget, the packaged RSSM has worse speed error than the GRU. This is a lesson in optimization and evidence, not a defect to hide.

# Chapter 52 - Evaluate the Held-Out Rollout

**WHAT YOU ARE LEARNING:** Inverse transforms, per-feature errors, horizon curves and physical checks.

**WHY IT MATTERS:** A normalized scalar loss is not interpretable to a race engineer.

**YOU ARE FINISHED WHEN:** You can report km/h error and identify where error compounds.




*The reference rollout error rises as predictions feed the next transition.*

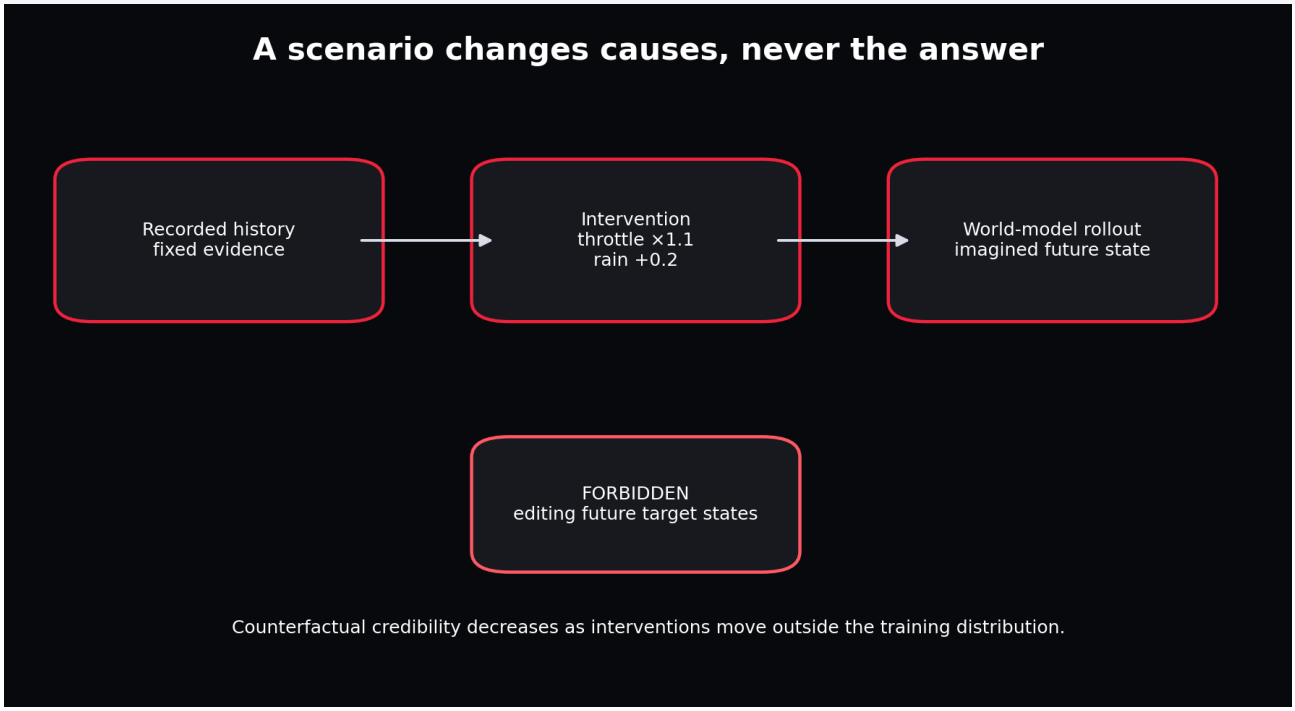
```
pred_raw = standardizer.inverse_targets(prediction_z)
error = pred_raw - target_raw
speed_mae = mean(abs(error[..., speed_index]))
horizon_rmse = sqrt(mean(error**2, axis=(batch, feature)))
```

# Chapter 53 - Scenario Interventions

**WHAT YOU ARE LEARNING:** Changing future actions/context while holding history fixed.

**WHY IT MATTERS:** Simulation requires changing causes, not editing outcomes.

**YOU ARE FINISHED WHEN:** You can create an intervention and audit exactly which columns changed.




*A valid scenario edits causes and lets the world model produce the consequence.*

```
scenario = Scenario(
    throttle_scale=1.10,
    brake_scale=0.95,
    grip_multiplier=0.90,
    rain_delta=0.20,
)
future_changed = apply_scenario(future_inputs_raw, scenario)
prediction = rollout(model, history_raw, future_changed, standardizer)
```

Intervention distance matters. The model may behave plausibly under small control changes and fail under extreme combinations. Compute feature-space distance from training data and flag extrapolative scenarios in future versions.

# Chapter 54 - Publish the First Simulation Artifact

**WHAT YOU ARE LEARNING:** Rollout CSV, metrics JSON, model metadata and summary.

**WHY IT MATTERS:** A prediction that exists only in memory cannot be reproduced or reviewed.

**YOU ARE FINISHED WHEN:** Every result can be traced to run ID, config, checkpoint, scaler and split.

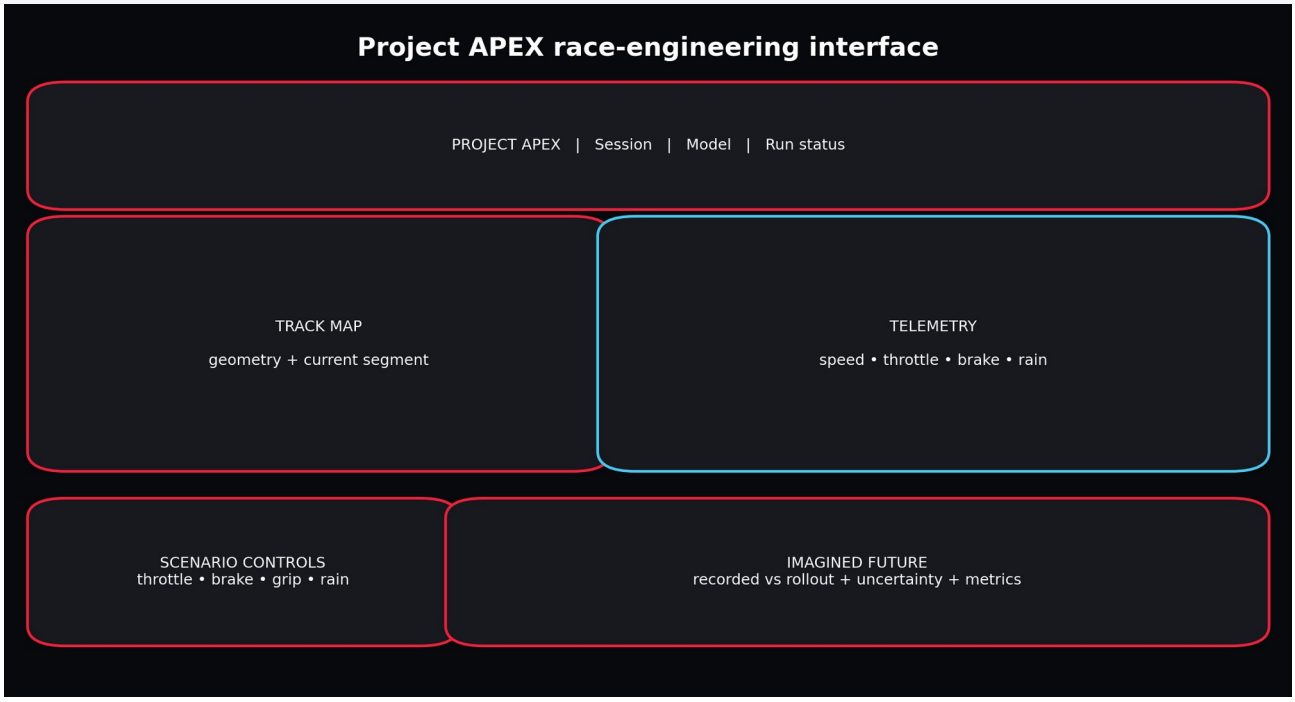
```
artifacts/runs/<run_id>/
  canonical_telemetry.csv
  quality_report.json
  splits.json
  standardizer.json
  model/world_model.pt
  model/model_meta.json
  training_history.json
  metrics.json
  ablations.json
  rollout_preview.csv
  summary.json
```

# Chapter 55 - Launch the Working UI

**WHAT YOU ARE LEARNING:** Race desk, telemetry explorer, rollout simulator, evaluation and ablation tabs.

**WHY IT MATTERS:** A usable interface turns model artefacts into an inspectable product.

**YOU ARE FINISHED WHEN:** You can launch the UI and explain why it does not contain training business logic.




*The interface is a replaceable client over stable artifacts and simulation functions.*

```
apexsim ui  --run-dir artifacts/runs/reference_gru  --config configs/fast.yaml
```

The Race Engineering Desk visualizes a session and track trace. The World Rollout Simulator changes controls and context. Model Evaluation shows horizon behaviour. Ablation Lab exposes evidence for feature choice. The architecture tab makes the data/model boundary visible.

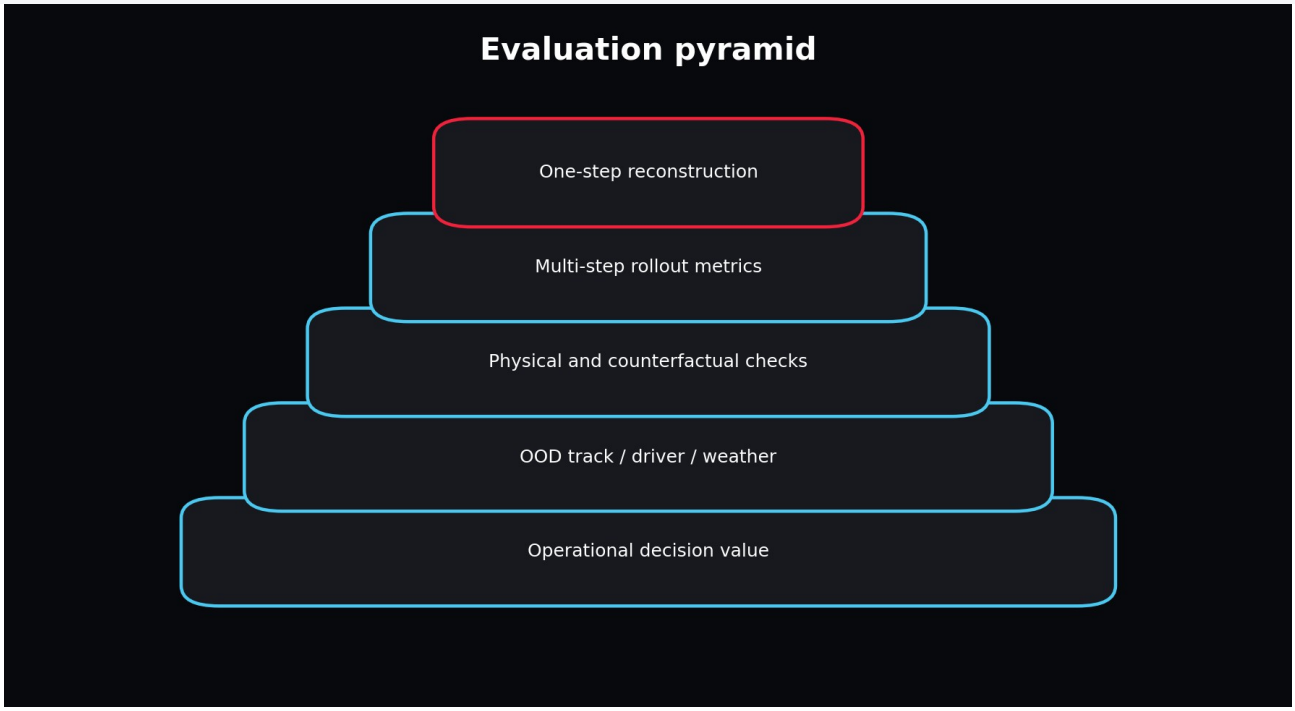
# Part VII - Evaluation, Ablations and Fine-Tuning

# Chapter 56 - The Evaluation Pyramid

**WHAT YOU ARE LEARNING:** From one-step reconstruction to operational decision value.

**WHY IT MATTERS:** Good one-step error is necessary but insufficient.

**YOU ARE FINISHED WHEN:** You can design a layered evaluation plan.




*Evaluation becomes more realistic and expensive as it moves upward.*

Start with feature error to debug. Add rollout curves to measure compounding. Add physical and counterfactual checks to reject impossible behaviour. Add out-of-distribution track/driver/weather holdouts. Finally ask whether the simulator improves a decision or controller.

# Chapter 57 - Metrics by Feature and Horizon

**WHAT YOU ARE LEARNING:** MAE, RMSE, normalized error and horizon aggregation.

**WHY IT MATTERS:** Different channels and time horizons fail differently.

**YOU ARE FINISHED WHEN:** You can choose metrics that reflect operational consequences.

| Metric | Use | Caution |
|---|---|---|
| MAE | Typical absolute error in real units | Treats all errors linearly |
| RMSE | Penalizes large misses | Sensitive to outliers |
| Normalized loss | Balanced training across channels | Hard to interpret operationally |
| Horizon curve | Shows compounding and useful range | Needs consistent sampling rate |
| Circular error | Track-progress angle | Must respect wraparound |

# Chapter 58 - Physical and Semantic Validity

**WHAT YOU ARE LEARNING:** Range checks, conservation-like constraints and representation validity.

**WHY IT MATTERS:** A low average error can coexist with impossible individual trajectories.

**YOU ARE FINISHED WHEN:** You can define model-specific invariants.

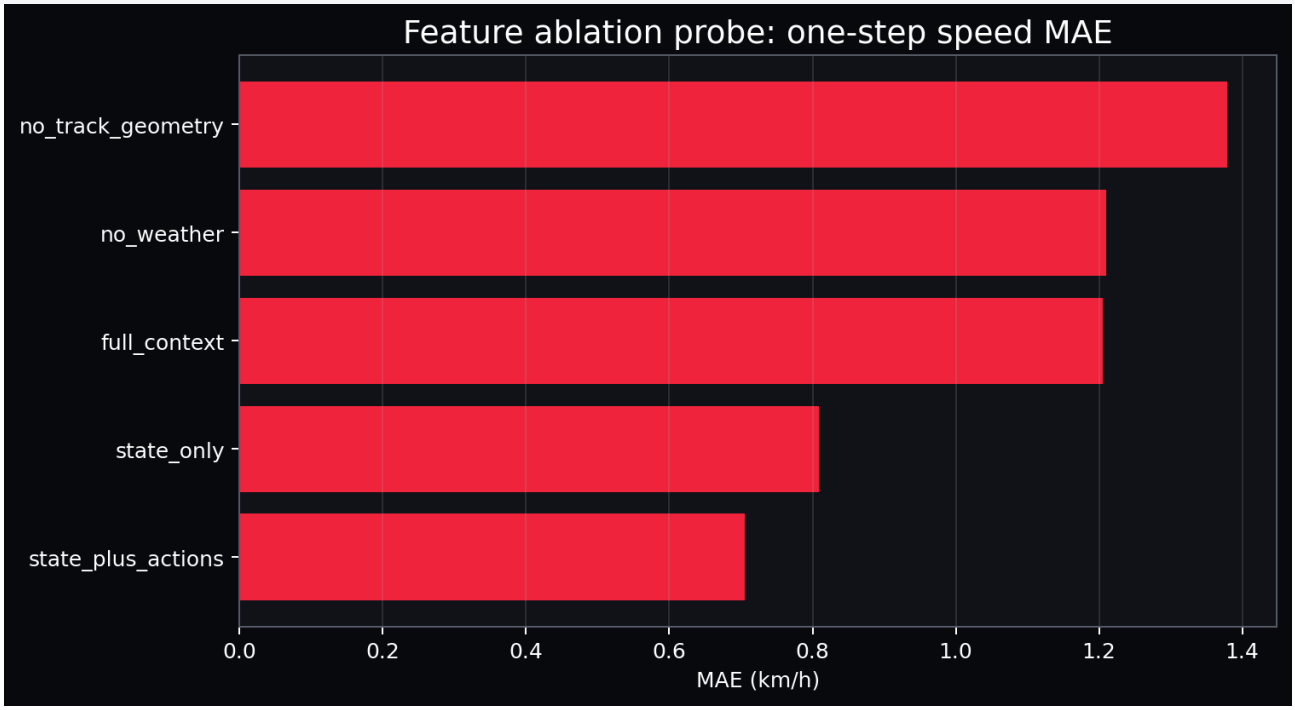
- Speed should not be negative or far beyond plausible bounds.
- Throttle/brake scenario inputs remain in [0,1].
- Progress sine/cosine should remain near unit norm.
- Tyre age should not decrease during an uninterrupted stint.
- Position/progress should move consistently with positive speed.
- Predictions should respond in the expected direction to small controlled interventions.

# Chapter 59 - Ablation Science

**WHAT YOU ARE LEARNING:** Control, treatment, held-fixed variables and decision criteria.

**WHY IT MATTERS:** Random hyperparameter changes do not reveal which information or mechanism matters.

**YOU ARE FINISHED WHEN:** You can write a complete experiment card before running code.




*The cheap reference probe compares information sets under one fixed linear model.*

| Ablation | Question |
|---|---|
| state_only | Can recent state carry the short transition without controls? |
| state_plus_actions | Do driver controls improve next-state prediction? |
| full_context | Does weather/geometry add measurable value? |
| no_weather | How much does environmental context contribute? |
| no_track_geometry | Does the model need where/what corner it is approaching? |

**INTERPRETATION:** An ablation result is conditional on model, horizon, split and data. A feature useless for one-step ridge prediction may be essential for long stochastic rollouts.

# Chapter 60 - Architecture Comparisons

**WHAT YOU ARE LEARNING:** Fair GRU, SSM and RSSM experiments.

**WHY IT MATTERS:** Different models should be compared under equivalent information and budgets.

**YOU ARE FINISHED WHEN:** You can prevent a misleading benchmark.

7. Use identical train/validation/test sessions.
8. Use the same feature order and scaler.
9. Match or report parameter counts.
10. Use the same maximum epochs and early-stopping protocol.
11. Report training/inference time and memory.
12. Evaluate deterministic and stochastic models appropriately.
13. Repeat across seeds if the decision matters.

# Chapter 61 - Pretraining Across Tracks

**WHAT YOU ARE LEARNING:** Self-supervised next-state learning over diverse sessions.

**WHY IT MATTERS:** A general dynamics prior can reduce data needed for a new circuit.

**YOU ARE FINISHED WHEN:** You can create pretrain and target-track split policies without leakage.

Pretrain on many track/session episodes. Hold the target circuit completely out. Then fine-tune using only target-track training sessions and evaluate on separate target-track sessions. Never pretrain on the target test session and call the result zero-shot.

# Chapter 62 - Fine-Tuning Strategies

**WHAT YOU ARE LEARNING:** Frozen backbone, head-only, adapters, partial unfreezing and full fine-tune.

**WHY IT MATTERS:** Target data volume and domain shift determine how much of the model should move.

**YOU ARE FINISHED WHEN:** You can choose a strategy and monitor forgetting.

| Strategy | Use when | Risk |
|---|---|---|
| Head-only | Backbone features transfer and target data is tiny | Insufficient adaptation |

| | | |
|---|---|---|
| Last layer/block | Moderate shift and limited data | Representation remains partly mismatched |
| Adapters/LoRA-like modules | Need parameter efficiency | Implementation complexity for custom recurrent models |
| Full fine-tune | Substantial target data or strong shift | Overfit and forget broad dynamics |
| Joint rehearsal | Need target adaptation without forgetting | Requires source replay data |

```
# Conceptual target-track fine-tune
model.load_state_dict(pretrained_checkpoint)
optimizer = AdamW(model.parameters(), lr=1e-4)
train_on(target_track_train_sessions)
evaluate_on(target_track_test_sessions)
```

# Chapter 63 - Adapting Existing Time-Series and Dreamer Models

**WHAT YOU ARE LEARNING:** Patch-based forecasters, time-series foundation models and Dreamer repositories.

**WHY IT MATTERS:** Reusing weights saves work only if input semantics and objective align.

**YOU ARE FINISHED WHEN:** You can write a compatibility checklist before downloading a large model.

| Compatibility question | Why it matters |
|---|---|
| Does the model accept multivariate covariates/actions? | Unconditional forecasts cannot represent control interventions |
| Is sampling frequency compatible? | Temporal scale changes dynamics |
| Can it predict multiple state channels jointly? | Independent forecasts may violate cross-feature physics |
| Can features be normalized exactly as expected? | Pretraining assumptions affect transfer |
| Is recurrent online inference available? | Simulation may require low-latency step-by-step rollout |
| Can uncertainty be calibrated? | Samples are not useful if coverage is wrong |

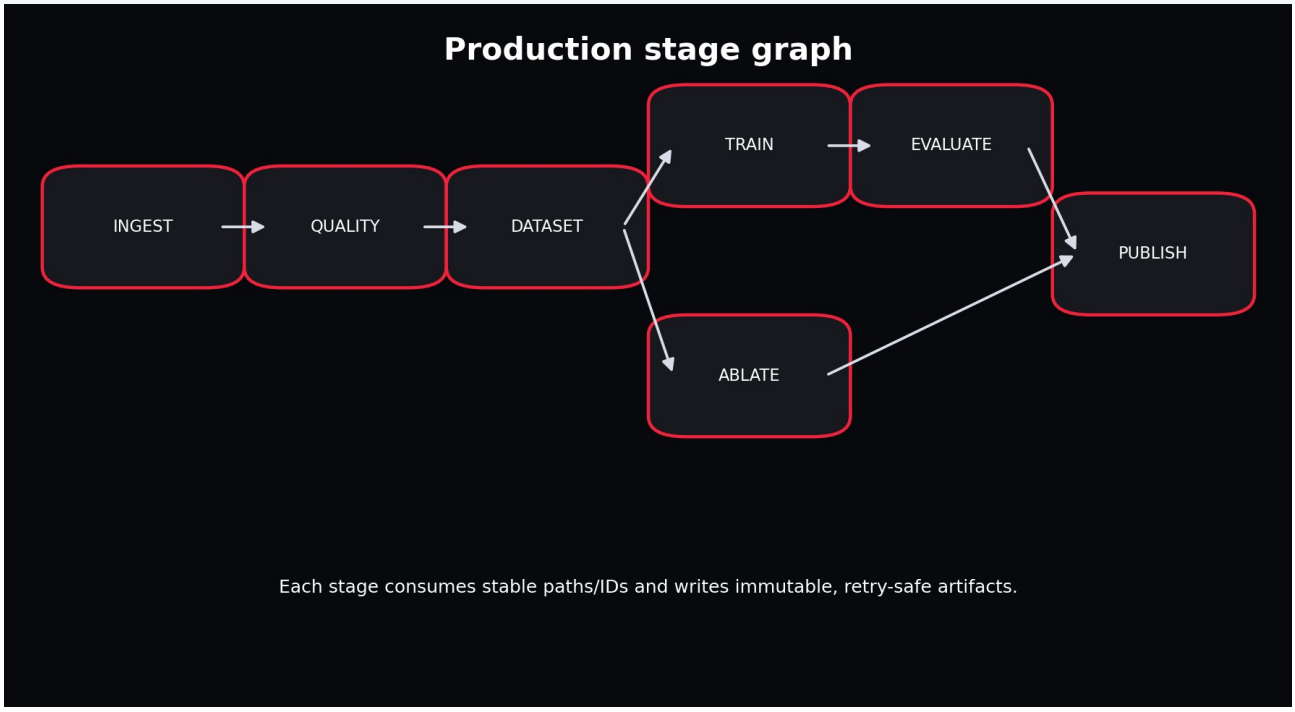
# Part VIII - Production, UI and Operations

# Chapter 64 - Idempotent Pipeline Stages

**WHAT YOU ARE LEARNING:** Path-based inputs, immutable outputs and safe retries.

**WHY IT MATTERS:** Production pipelines fail and rerun; duplicate side effects are unacceptable.

**YOU ARE FINISHED WHEN:** You can rerun a completed stage without changing its output.




*The local runner and Airflow wrapper call the same stage functions.*

Each stage checks for its expected artifact. Ingestion produces canonical data. Quality produces a report. Dataset stage produces split and scaler artifacts. Training writes a checkpoint and metadata. Evaluation and ablation write results. Publication writes rollout artifacts and summary. In a larger system, use content-addressed or versioned paths instead of silently overwriting.

# Chapter 65 - Run Registry and Lineage

**WHAT YOU ARE LEARNING:** Run IDs, states, timestamps, metrics and artifact locations.

**WHY IT MATTERS:** Without lineage, a good result cannot be trusted or reproduced.

**YOU ARE FINISHED WHEN:** You can answer which code/config/data/model produced a displayed rollout.

SQLite is appropriate for a single-user local V1. The registry records running, succeeded or failed status, model type, start/finish time, directory and metrics. A multi-user production system would move to a server database or experiment platform with transactions and access control.

# Chapter 66 - Airflow Orchestration

**WHAT YOU ARE LEARNING:** DAGs, tasks, retries, scheduling and thin orchestration.

**WHY IT MATTERS:** Scheduled ingestion and retraining should reuse tested business logic.

**YOU ARE FINISHED WHEN:** The DAG imports safely and task bodies call project stages rather than reimplementing them.

```
@dag(schedule=None, catchup=False)
def project_apex_world_model():
    @task(retries=2)
    def run_pipeline_task(config_path, run_id):
        return run_pipeline(load_config(config_path), run_id)
    run_pipeline_task("configs/fast.yaml", "airflow_scheduled_run")
```

**SCALE-UP DESIGN:** For real scale, split stages into separate tasks, pass only paths/IDs through XCom, use pools for API rate limits and GPU resources, and make backfills explicit.

# Chapter 67 - Testing Strategy

**WHAT YOU ARE LEARNING:** Unit, contract, model-shape, scenario and end-to-end tests.

**WHY IT MATTERS:** ML systems fail in ordinary software layers as often as in model mathematics.

**YOU ARE FINISHED WHEN:** You can state which regression each included test prevents.

| Test | Protects |
|---|---|
| Synthetic contract | Generator and schema compatibility |

| | |
|---|---|
| Window shapes/splits | Axis semantics and episode leakage |
| Model shapes | Architecture interface compatibility |
| Scenario edit | Counterfactual changes only intended features |
| End-to-end pipeline | Stages, artifacts, registry and training integration |

# Chapter 68 - Monitoring and Drift

**WHAT YOU ARE LEARNING:** Data, feature, model, physical and product health.

**WHY IT MATTERS:** A model can remain online while its assumptions decay.

**YOU ARE FINISHED WHEN:** You can separate drift detection from automatic retraining.

| Layer | Signals |
|---|---|
| Pipeline | freshness, failures, runtime, retry count |
| Data | schema, missingness, timestamp gaps, source changes |
| Feature | mean/std/quantile shift, unseen tracks/weather |
| Model | horizon error, calibration, physical violations |
| Product | UI/API latency, failed scenarios, user feedback |

**RETRAINING CAUTION:** Drift is a diagnosis signal. Automatic retraining without labels, evaluation and promotion gates can deploy a worse model faster.

# Chapter 69 - FastAPI Service Boundary

**WHAT YOU ARE LEARNING:** Health, runs, run details and rollout artifacts.

**WHY IT MATTERS:** Interfaces should expose stable contracts instead of internal Python objects.

**YOU ARE FINISHED WHEN:** You can replace the UI without touching model code.

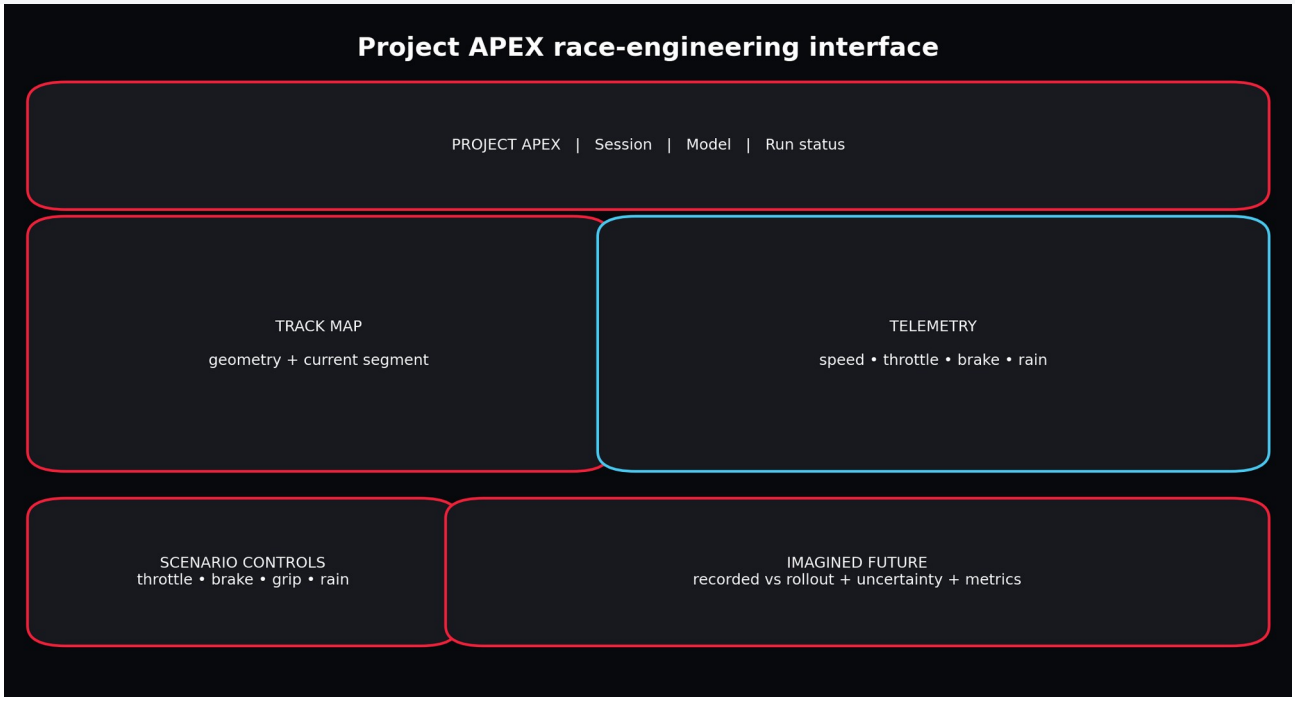
```
GET /health
GET /runs
GET /runs/{run_id}
GET /runs/{run_id}/rollout
```

# Chapter 70 - Designing a Beautiful, Useful UI

**WHAT YOU ARE LEARNING:** Information hierarchy, race-engineering workflows and progressive disclosure.

**WHY IT MATTERS:** A beautiful interface is not decoration; it helps users distinguish observation, model belief and scenario choice.

**YOU ARE FINISHED WHEN:** You can identify what belongs on each tab and why.




*The layout separates evidence, controls and imagined output.*

- Use one dominant question per tab.
- Label recorded data and imagined data differently.
- Display units everywhere.
- Make model/run identity visible.
- Show horizon and uncertainty, not only a final number.
- Prevent extreme unsupported interventions or warn about them.
- Keep raw diagnostic artifacts available for engineers.

# Part IX - F1 25 Migration and Long-Term Roadmap

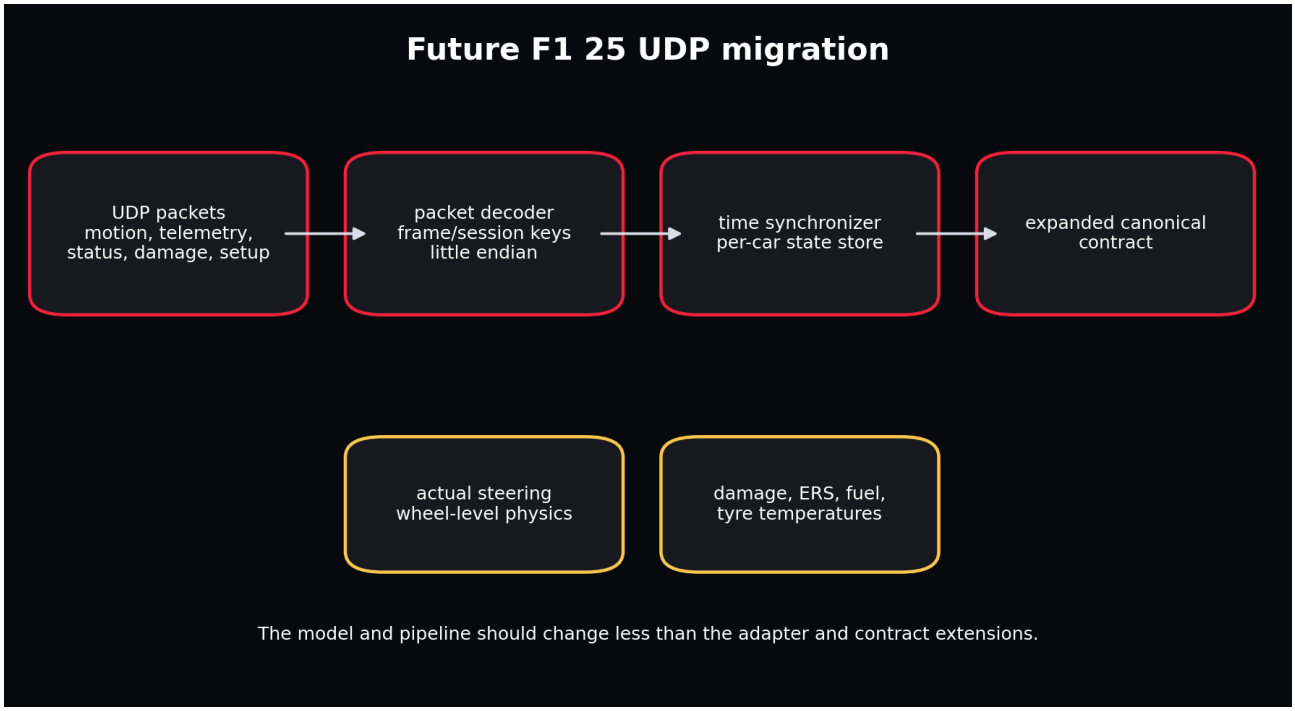
# Chapter 71 - Understanding the F1 25 UDP Feed

**WHAT YOU ARE LEARNING:** Packets, frame/session identifiers, send rate, coordinates, privacy restrictions and packed binary structures.

**WHY IT MATTERS:** The game feed transforms APEX from action-poor historical modelling into richer closed-loop telemetry.

**YOU ARE FINISHED WHEN:** You can outline a robust listener and synchronization layer.

The official F1 25 specification describes packet types for motion, session, lap data, participants, car setups, telemetry, status, damage, tyre sets, extended motion, time trial and lap positions. UDP output can be configured in game, commonly on port 20777 with a selectable send rate. Structures are packed and little-endian; vehicle indices remain stable within a session. Some fields can be restricted in multiplayer privacy settings.




*The adapter expands while the downstream model/pipeline interfaces remain largely stable.*

**NETWORK REALITY:** UDP is unordered and unreliable. Use packet sequence/frame identifiers, detect drops and reordering, and never assume arrival order is perfect.

# Chapter 72 - Expanded F1 25 Contract

**WHAT YOU ARE LEARNING:** Motion, controls, wheel state, damage, setup, energy and race state.

**WHY IT MATTERS:** Richer sensors enable a more Markov and physically meaningful world state.

**YOU ARE FINISHED WHEN:** You can prioritize features by causal value and availability.

| Packet family | Potential APEX features |
|---|---|
| Motion | world position/velocity, direction vectors, G-forces, yaw/pitch/roll |
| Car telemetry | speed, throttle, steer, brake, clutch, gear, RPM, temperatures, pressures |
| Car status | fuel, ERS, compounds, DRS permission, traction/ABS settings |
| Damage | wing/floor/diffuser/sidepod/engine/gearbox/tyre wear and damage |
| Setup | wing, differential, suspension, brakes, pressures |
| Session/lap | track, weather, safety car, sector, pit and race state |

# Chapter 73 - From Offline World Model to Closed-Loop Simulator

**WHAT YOU ARE LEARNING:** UDP recording, replay, online inference, shadow mode and controller integration.

**WHY IT MATTERS:** Closed-loop use introduces latency, distribution shift and safety concerns.

**YOU ARE FINISHED WHEN:** You can design a staged deployment rather than connecting an untested policy directly.

14. Record raw packets with timestamps and packet identifiers.
15. Build deterministic replay and compare decoded state with in-game displays.
16. Run world-model inference in shadow mode without controlling anything.
17. Measure real-time latency and packet-loss resilience.

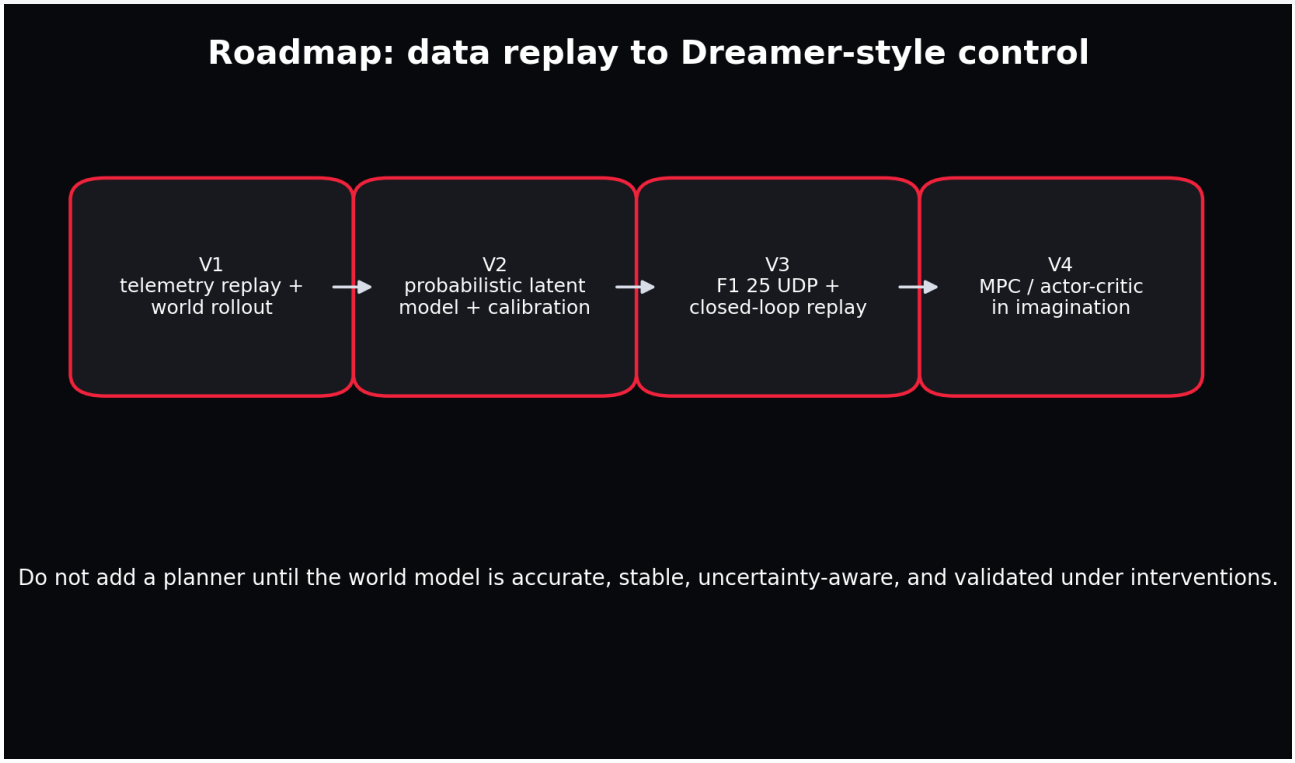
18. Validate short rollouts against subsequent real frames.
19. Add MPC suggestions while a human remains in control.
20. Only then test limited automated control in a safe environment.

# Chapter 74 - Roadmap to Dreamer for F1 25

**WHAT YOU ARE LEARNING:** Progressive milestones from telemetry replay to imagined control.

**WHY IT MATTERS:** The ultimate vision needs ordered evidence gates.

**YOU ARE FINISHED WHEN:** You can explain why each milestone must pass before the next.




*The path to actor-critic control begins with trustworthy replay and dynamics.*

V1 establishes data contracts and deterministic rollout. V2 adds calibrated stochastic latent dynamics. V3 integrates real game actions and richer state through UDP and validates online replay. V4 adds MPC or Dreamer-style actor/critic learning in imagination. At every step, retain a simple baseline and closed-loop reality check.

# Part X - Independent Mastery and Reference

# Chapter 75 - The Complete Instructor-Led Build Sequence

> **WHAT YOU ARE LEARNING:** A literal order for completing the project.
>
> **WHY IT MATTERS:** The sequence prevents model excitement from skipping data and evaluation foundations.
>
> **YOU ARE FINISHED WHEN:** Every phase has an artifact, a deliberate failure and a decision record.

21. Write V1 scope and non-goals.
22. Set up environment and run tests.
23. Generate synthetic telemetry.
24. Inspect units and one causal trajectory.
25. Validate the canonical contract.
26. Break ranges and duplicate keys.
27. Create deterministic session splits.
28. Fit and persist train-only normalization.
29. Build sequence windows.
30. Implement persistence and ridge baselines.
31. Trace GRU shapes.
32. Overfit one batch.
33. Train full fast GRU.
34. Evaluate by feature and horizon.
35. Run scenario interventions.
36. Run feature ablations.
37. Train SSM challenger.
38. Train RSSM challenger and inspect KL.
39. Register and publish artifacts.
40. Launch UI and API.
41. Add a real FastF1/OpenF1 adapter run.
42. Pretrain across tracks and fine-tune a held-out target.
43. Design F1 25 packet adapter.
44. Rebuild core components without viewing source.

# Chapter 76 - Common Failure Atlas

**WHAT YOU ARE LEARNING:** Symptoms, likely causes and minimum diagnostic steps.

**WHY IT MATTERS:** Knowing where to look is more valuable than trying random hyperparameters.

**YOU ARE FINISHED WHEN:** You can isolate a failure to data, optimization, model, evaluation or product layer.

| Symptom | Likely cause | First check |
|---|---|---|
| Near-perfect test score | Temporal leakage or future feature | Print session IDs and audit feature availability |
| Loss is NaN | bad values, exploding gradients, invalid std | Inspect batch min/max/finite and loss components |
| Training improves, rollout fails | exposure bias/compounding | Plot horizon curve and overfit autoregressive objective |
| Scenario has no effect | action not used or overwritten | Compare modified input columns and gradients |
| RSSM reconstructs but imagines badly | prior/posterior mismatch | Track KL and prior-only rollout |
| UI crashes after retrain | artifact/schema mismatch | Validate model metadata and scaler feature order |
| Real data much worse | domain shift/adapter error | Compare units, cadence and feature distributions |

# Chapter 77 - Exam-Style Questions

45. Why is a random telemetry-frame split usually invalid?
46. Explain the difference between state, action and context using five APEX columns.
47. Why encode track progress with sine and cosine?
48. What does `loss.backward()` compute conceptually?
49. Why can an RSSM posterior not be used during future imagination?
50. What benefit does a selective SSM offer over a fixed linear recurrence?
51. Why is a future steering proxy potentially leaky?
52. Design an ablation to test whether weather matters at a 5-second horizon.
53. What artifacts are required to reproduce a checkpoint prediction?
54. Why should an actor not be trained against an inaccurate world model?
55. How would you fine-tune a cross-track model on Monza without leaking the Monza test session?
56. Which F1 25 packets would most improve the Markov state and why?

# Chapter 78 - Solutions to Exam-Style Questions

1. Adjacent frames are highly correlated and often nearly identical; random splitting puts the same episode in train and test. Hold out complete sessions/events/tracks.

2. State: speed, acceleration, tyre age. Action: throttle, brake. Context: curvature, rain. The distinction is whether it is world condition, controller choice or predicted condition.

3. Raw progress has a discontinuity from 1 to 0 at the finish line. Sine/cosine preserves circular proximity.

4. It applies the chain rule through the computation graph to calculate the gradient of loss with respect to trainable parameters.

5. The posterior requires the actual future observation. At imagination time that observation is unknown, so the predictive prior must generate the latent.

6. Input-dependent write/forget behaviour can retain rare important events while maintaining a compact recurrent state and linear sequence scaling.

7. If derived from the realized future path, it contains information about what actually happened rather than only what was known/planned at prediction time.

8. Train identical models/splits/budgets with full context and with weather columns removed; compare horizon-specific error and wet-session subsets across seeds.

9. Code/model architecture, checkpoint, feature order, units, standardizer, config, data version, split, run ID and inference/scenario parameters.

10. The actor can discover and exploit inaccuracies, maximizing imagined reward while failing in the real environment.

11. Pretrain without any Monza sessions, fine-tune on a Monza training subset, choose checkpoints on separate Monza validation, and report on sealed Monza test sessions.

12. Actual steering and motion improve action/state; telemetry adds pedal/wheel temperatures; status/setup/damage add hidden causes affecting dynamics and long-term outcomes.

# Chapter 79 - Code Atlas

| File | Responsibility |
|---|---|
| config.py | Validate experiment and runtime intent |
| contracts.py | Define stable source-independent semantics |
| data/synthetic.py | Generate causal offline telemetry |
| data/fastf1_adapter.py | Translate FastF1 session data |
| data/openf1_adapter.py | Join and translate OpenF1 streams |
| data/validate.py | Enforce quality invariants |
| data/features.py | Fit and persist train-only scaling |

| | |
|---|---|
| data/windows.py | Create episode-safe sequence examples |
| models/gru_world_model.py | Primary deterministic dynamics |
| models/rssm.py | Stochastic Dreamer-style dynamics |
| models/ssm_world_model.py | Selective state-space challenger |
| training.py | Optimize and checkpoint parameters |
| evaluation.py | Compute horizon and physical metrics |
| simulation.py | Apply controlled future interventions |
| pipeline/stages.py | Idempotent business stages |
| pipeline/runner.py | Local state machine and registry updates |
| ui.py | Interactive client |
| serving.py | REST inspection service |

# Chapter 80 - References and Further Study


57. FastF1 official documentation and repository: access to F1 timing, telemetry, position, tyre, weather and results data; documentation version observed during preparation: 3.8.1.
58. OpenF1 official API documentation: historical telemetry/timing endpoints from 2023 onward, including car data, location, laps, weather, stints and session data.
59. EA SPORTS, Data Output from F1 25 Game, UDP specification version 3: packet structures, telemetry configuration, coordinate conventions and packet families.
60. Hafner, Pasukonis, Ba, Lillicrap. Mastering Diverse Domains through World Models (DreamerV3), arXiv:2301.04104 and official repository.
61. Gu, Goel, Re. Efficiently Modeling Long Sequences with Structured State Spaces (S4), arXiv:2111.00396.
62. Gu and Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces, arXiv:2312.00752.
63. Nie et al. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers (PatchTST).
64. Ansari et al. Chronos: Learning the Language of Time Series, TMLR 2024 and official Amazon Science repository.


> **FINAL STANDARD:** You have mastered Project APEX when you can build a different action-conditioned telemetry simulator from scratch, explain every axis and artifact, design an honest evaluation, and know exactly what evidence is missing before making stronger claims.

# Appendix A - Key Commands

```
# Verify
pytest -q
python -m py_compile dags/apexsim_dag.py

# Build a complete offline run
apexsim run --config configs/fast.yaml --run-id reference_gru

# Compare architectures
apexsim run --config configs/ssm_fast.yaml --run-id reference_ssm
apexsim run --config configs/rssm_fast.yaml --run-id reference_rssm

# Launch product surfaces
apexsim ui --run-dir artifacts/runs/reference_gru --config configs/fast.yaml
apexsim api --artifacts-dir artifacts/runs

# Optional historical sources
apexsim ingest-fastf1 --year 2025 --event Monza --session R --driver VER
apexsim ingest-openf1 --session-key 9159 --driver-number 55
```

# Appendix B - Verified Reference Run

| Item | Verified value |
|---|---|
| Canonical rows | 8,400 |
| Sessions | 8 |
| Model | GRU |
| Input features | 16 |
| Predicted state features | 5 |
| Held-out windows | 1200 |
| Speed MAE | 13.71 km/h |
| Speed RMSE | 20.42 km/h |
| Negative-speed rate | 0.0 |
| Best cheap ablation | state_plus_actions |

**INTERPRETATION:** This is a small synthetic educational benchmark. It proves that the system works end to end; it does not establish real F1 predictive accuracy.